

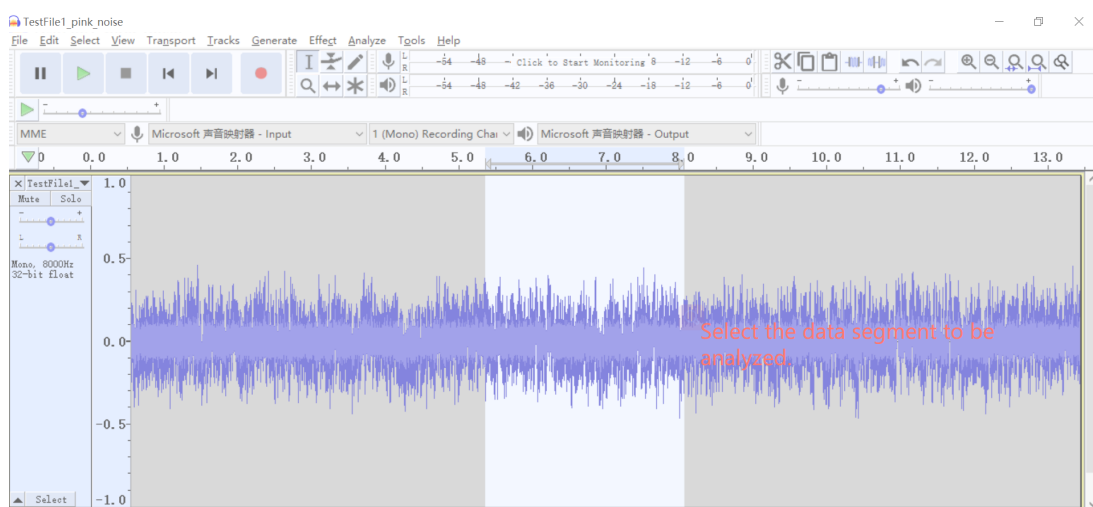
SSD_Audio debugging instructions

1. Common operations of AUDACITY

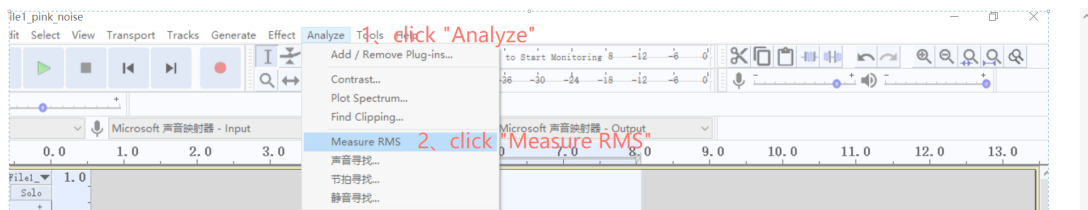
1.1. View gain

During the debugging of the intercom, it is often necessary to check the signal size over a period of time. The following figure is used as an example to illustrate how to check the size of the signal through Adobe audition.

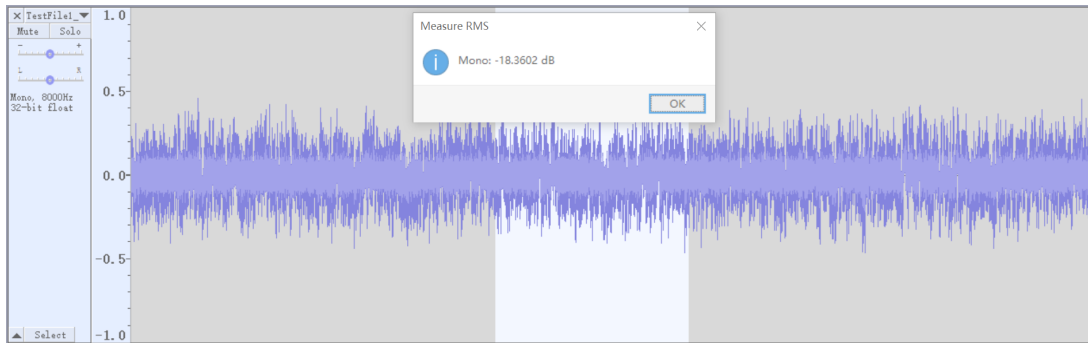
1. Select the time period to be analyzed



2. Open the Analyze window



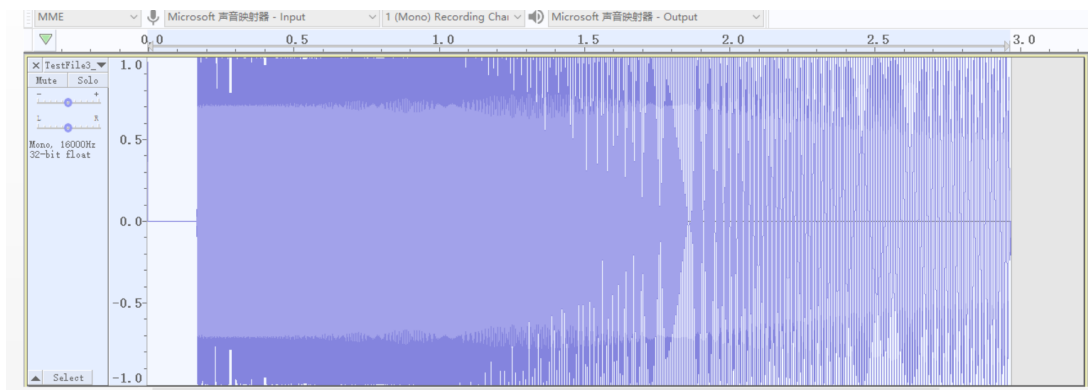
3. View the average amplitude of the selection



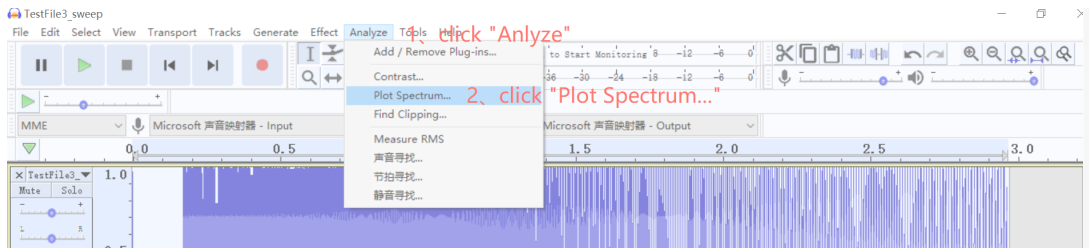
1.2. View frequency response characteristics

During the debugging of the intercom, it is often necessary to adjust the frequency response characteristics of the device. The following will illustrate how to view the frequency response characteristics of the audio file through Audacity. When we get the frequency response curve of the current device, we can adjust it through EQ.

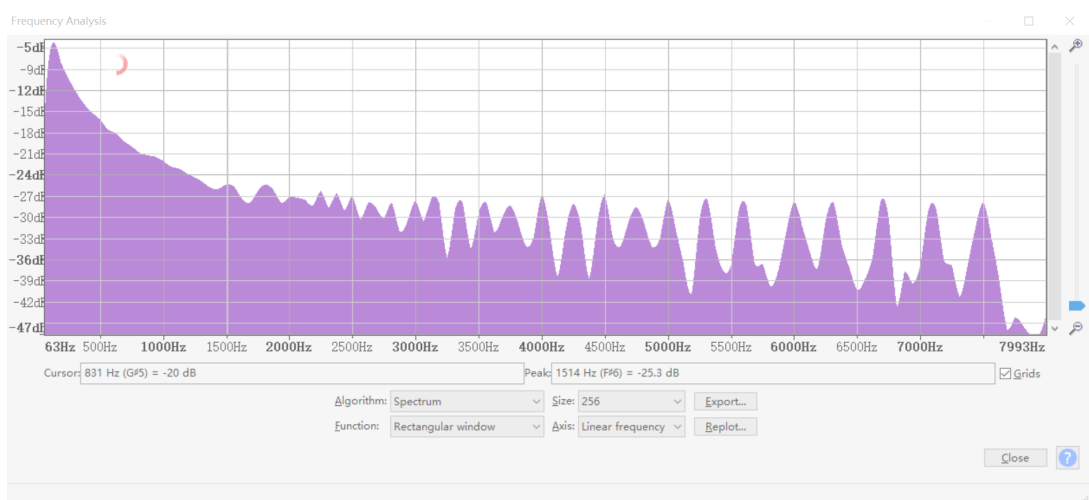
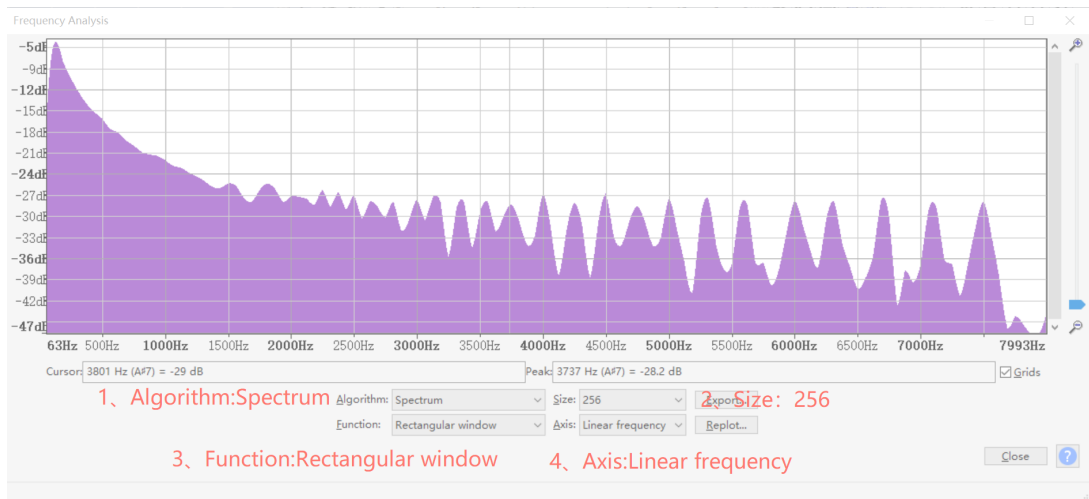
1. Select the data selection to analyze



2. Open the Analyze window



3. Click "Scan Selection"



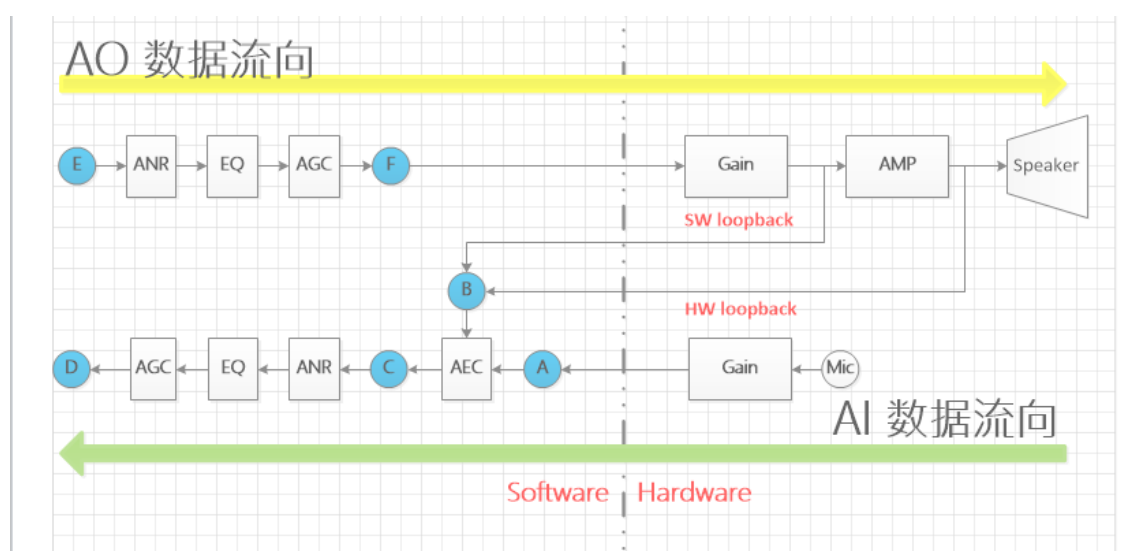
2. Introduction to the internal process of MI Audio

2.1. MI Audio Internal Block Diagram

Note: When the MI_AI version is 2.19 and the MI_AO version is 2.17, the MI no longer includes the implementation of the algorithm and the concatenation process of the algorithm (including the debug and dump data provided by the original MI), but it is still recommended to use the following process.
Concatenation algorithm.

The following figure shows the basic data flow of MI Audio:

(The following description is the basic situation that does not include sound source localization, sound detection, beamforming, resampling, codec and other algorithms)



2.2. Ai data flow analysis

上图中绿色箭头方向为AI数据的流向，数据由麦克风采集，经过芯片内部的增益，到达MI层（A点），此处（A点）的数据为原始的PCM数，接着和AO端的数据（B点）一起送入AEC算法进行处理，AEC算法处理后的数据（C点）会继续送到后面的APC算法（APC为ANR、EQ、HPF、AGC算法的集合）进行处理，处理后的数据即为最终的数据（D点）。

SW loop back与HW loop back的区别：

主要区别在于送回AEC算法的AO数据差异。

SW loop back用于处理只经过了芯片内部增益的AO数据，好处在于不需要外部电路进行特定分压设计，且只需占用一个音频输入通道，与HW loop back相比，对于外部电路的功放芯片引起的失真，SW loop back没法较好地进行处理。

HW loop back用于处理外部电路功放芯片处理后的AO数据，可以较好地应对上述情况，但需要对外部电路进行特定的分压设计，多占用一个音频输入通道，以及需要确保loop back回来的峰值不能超过芯片内部量程（3.3V）。HW loop back必须要有两个音频输入通道才能使用。

注意：HW loop back 只有两通道的Amic才能使用。

2.3. Ao数据流向分析

上图中黄色箭头方向为AO数据的流向，待播放PCM数据（E点）由应用通过MI接口送入后，经过APC算法（ANR、EQ、AGC处理后（F点）），经过芯片内部的增益、外部电路的功放芯片，再由喇叭输出。

2.4. 确认原始数据和算法处理正常

上图中所有蓝色点的数据都可以通过SDK提供的debug手段dump出来（因为此功能仅仅是为了debug用，若有出现文件名有误，SDK暂时不做修正）。SDK提供的dump手段，需要在应用起来之前，通过export一系列的环境变量来指定文件的保存路径以及需要dump的数据位置。

- 指定文件保存的路径

```
export MI_AI_DUMP_PATH=xxx，指定AI的dump路径为xxx
```

```
export MI_AO_DUMP_PATH=xxx，指定AO的dump路径为xxx
```

- 指定dump的数据位置

AI:

```
export MI_AI_DUMP_PCM_ENABLE=1，指定dump AI的原始pcm（即A点处的数据），文件名为MI_AI_Dev*Chn*_*K_Src.pcm。
```

`export MI_AI_DUMP_AEC_ENABLE=1`，指定dump AI AEC前后的数据（即A、B、C点处的数据），文件名为MI_AI_Dev*Chn*_K_AiIn.pcm、MI_AI_Dev*Chn*_K_AoIn.pcm和MI_AI_Dev*Chn*_K_AecOut.pcm。（当AEC开启时，MI_AI_Dev*Chn*_K_AiIn.pcm和MI_AI_Dev*Chn*_K_Src.pcm的内容是一致的。）

`export MI_AI_DUMP_VQE_ENABLE=1`，指定dump AI APC前后的数据（C、D点处的数据），文件名为MI_AI_Dev*Chn*_K_VqeIn.pcm和MI_AI_Dev*Chn*_K_VqeOut.pcm（当AEC开启时，MI_AI_Dev*Chn*_K_VqeIn.pcm和MI_AI_Dev*Chn*_K_AecOut.pcm的内容是一致的。当没有开启AEC时，MI_AI_Dev*Chn*_K_Src.pcm和MI_AI_Dev*Chn*_K_VqeIn.pcm的内容是一致的。）

AO:

`export MI_AO_DUMP_PCM_ENABLE=1`，指定dump 最终送给驱动的数据（即F点处的数据），文件名为MI_AO_Dev*_K_Dst.pcm。

`export MI_AO_DUMP_VQE_ENABLE=1`，指定dump AO APC前后的数据（即E、F点处的数据），文件名为MI_AO_Dev*_K_VqeIn.pcm和MI_AO_Dev*_K_VqeOut.pcm。（当AO端只有APC算法开启时，MI_AO_Dev*_K_VqeOut.pcm和MI_AO_Dev*_K_Dst.pcm的内容一致。）

(*为应用具体使用的device id、channel id、采样率)

建议只dump想要的节点处的音档，不要所有都同时dump，不然可能会引起一直在写文件导致SDK丢帧。

3. Audio 相关算法介绍

3.1. AEC (Acoustic Echo Cancellation)

3.1.1. AEC算法的简单介绍

AEC即回声消除，通过算法来消除由扬声器设备发出的直接或间接耦合回麦克风的语音。

3.1.2. AEC算法的参数说明

下面为MI API开给应用设定AEC参数的数据结构：

```
typedef struct
MI_AI_AecConfig_s
{
    MI_BOOL bComfortNoiseEnable;

    MI_S16 s16DelaySample;

    MI_U32 u32AecSupfreq[6];

    MI_U32 u32AecSupIntensity[7];

    MI_S32 s32Reserved;
} MI_AI_AecConfig_t;
```

下面为AEC算法开给应用设定AEC参数的数据结构：

```
typedef struct
{
    IAA_AEC_BOOL comfort_noise_enable;

    short delay_sample;

    unsigned int suppression_mode_freq[6];

    unsigned int suppression_mode_intensity[7];
}AudioAecConfig;
```

参数	作用	限制/注意事项/详细说明
bComfortNoiseEnable(MI)/ comfort_noise_enable(AEC)	AEC加入舒适噪声使能	默认情况设置FALSE。 说明：当使能该参数时，AEC算法的处理结果会加入一些背景噪声，使处理后声音不会过于空洞。此项主要是为了解决AEC消得太干净，导致ANR收敛需要太长时间（具体现象为语音之间存在噪声被拉起后又被压制的现象）。
s16DelaySample(MI)/ delay_sample(AEC)	左右声道间的回声延迟	默认情况设置0。 说明：由于麦克风和喇叭的放置位

	样本	置、麦克风间的距离等会造成左右声道接收到回声的时间点不一致，两个声道之间回声延迟有差异。此值表示左声道比右声道提早多少个采样点收到回声。 注意：此参数只适用于双声道做AEC的情况，且不建议使用。
u32AecSupfreq(MI)/ suppression_mode_freq(AEC)	AEC处理的 频段	说明：该数组将当前采样率的最高频率分成8个频段来处理。以下数组元素和频率范围的转换公式参见公式1 [#jump]。假设当前采样率16K，以{4, 6, 36, 49, 50, 51}为例，当前采样率为16K，最大的采样频率为8K，第一个频段F1 = (0~4) * 8000 / 128 = 0~250Hz，第二个频段F2 = (4~6) * 8000 / 128 = 250~375Hz，第三个频段F3 = (6~36) * 8000 / 128 = 375~2250Hz，第四个频段F4 = (36~49) * 8000 / 128 = 2250~3062Hz，第五个频段F5 = (49~50) * 8000 / 128 = 3062~3125Hz，第六个频段F6 = (50~51) * 8000 / 128 = 3125~3187Hz，第七个频段F7 = (51~128) * 8000 / 128 = 3187~8000Hz。 注意：数组要求每一个元素都必须比前一个元素大。 参数范围是[1,127]。
u32AecSupIntensity(MI)/ suppression_mode_intensity(AEC)	AEC处理的 强度	说明：此数组代表频段的回声消除强度，各元素和u32AecSupfreq划分出来的频率范围一一对应。 注意：参数的范围是[0,15]，强度越大，消除的细节越多，声音越不自然。反之，强度越小，细节保留越多，但会回声消除不够干净。（若对消除效果不满意，只能让用户来权衡，再做微调）

公式1

$$\frac{\text{residual} \setminus \text{echo} \setminus \text{frequency} \setminus \text{range}}{\frac{\text{sample} \setminus \text{rate}}{2}} \times \text{point number}(128)$$

3.2. ANR (Acoustic Noise Reduction)

3.2.1. ANR算法的简单介绍

ANR即声音降噪，通过算法来消除环境中持续存在的噪声（比如白噪声等），没法很好地处理突然出现的噪声。

3.2.2. ANR算法的参数说明

下面为MI API开给应用设定ANR参数的数据结构：

因NR算法有更新，请按照当前SDK版本对应的NR数据结构来相对应。

```
typedef struct
MI_AUDIO_AnrcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_U32    u32NrIntensity;

    MI_U32    u32NrSmoothLevel;

    MI_AUDIO_NrSpeed_e eNrSpeed;
}MI_AUDIO_AnrcConfig_t;
```

参数	作用	限制/注意事项/详细说明
eMode	ANR算法的模式	说明：此参数表示ANR算法的模式，不同的模式对应不同的应用场景。 E_MI_AUDIO_ALGORITHM_MODE_DEFAULT：此模式为语音模式，会使用算法中为语音数据调教的默认降噪参数。 E_MI_AUDIO_ALGORITHM_MODE_USER：此模式全部使用应用下的降噪参数。 E_MI_AUDIO_ALGORITHM_MODE_MUSIC：此模式为音乐模式，会使用算法中为音乐数据调教的降噪参数。（推荐使用此模式）
u32NrIntensity	ANR算法的强度	推荐值：20 说明：该参数表示ANR算法的降噪强度，该值越大，降噪效果越好，但细节损失越多。反之，降噪强度越小，细节保留越多，但噪声也没法去除干净。（此值也需要权衡和微调。） 注意：参数范围为 [0,30]。
u32NrSmoothLevel	ANR算法的平滑程度	推荐值：10 说明：该参数表示ANR算法处理的平滑程度，值越大降噪效果越平滑。平滑程度越小在频域上损失的细节越多。 注意：参数范围为 [0,10]。
eNrSpeed	ANR算法的收敛速度	推荐值：E_MI_AUDIO_NR_SPEED_MID 说明：此值表示ANR算法的收敛速度，收敛速度越快损失的细节越多。反之，收敛速度越慢，参考的时间越长，细节保留越多，但也会延长降噪起作用的时间。当当前的收敛速度已设置为E_MI_AUDIO_NR_SPEED_HIGH，但仍然收敛得比较慢时，可以打开AEC的bComfortNoiseEnable，让AEC的处理结果主动加入噪声，来辅助ANR的收敛。

```
typedef struct MI_AUDIO_AnrcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_U32    u32NrIntensityBand[_NR_BAND_NUM - 1];

    MI_U32    u32NrIntensity[_NR_BAND_NUM];
}
```

```
MI_U32    u32NrSmoothLevel;  
  
MI_AUDIO_NrSpeed_e eNrSpeed;  
  
}MI_AUDIO_AnrcConfig_t;
```

下面为ANR算法开给应用设定ANR参数的数据结构(仅说明与效果相关的部分)：

```
typedef struct{  
  
    unsigned int anr_enable;  
  
    unsigned int user_mode;  
  
    int anr_intensity_band[_NR_BAND_NUM-1];  
  
    int anr_intensity[_NR_BAND_NUM];  
  
    unsigned int anr_smooth_level;  
  
    NR_CONVERGE_SPEED anr_converge_speed;  
  
}AudioAnrConfig;
```

参数	作用	限制/注意事项/详细说明
eMode(MI)/ user_mode(ANR)	ANR算法的 模式	说明：此参数表示ANR算法的模式，不同的模式对应的应用场景。 E_MI_AUDIO_ALGORITHM_MODE_DEFAULT(MI)/0(此模式为语音模式，会使用算法中为语音数据调教的降噪参数。 E_MI_AUDIO_ALGORITHM_MODE_USER(MI)/1(ANR模式全部使用应用下的降噪参数。 E_MI_AUDIO_ALGORITHM_MODE_MUSIC(MI)/2(AN模式为音乐模式，会使用算法中为音乐数据调教的降噪参数。(推荐使用此模式)
u32NrIntensityBand(MI)/ anr_intensity_band(ANR)	ANR的频段 划分	说明：u32NrIntensityBand(MI)/anr_intensity_band(需要与u32NrIntensity搭配使用，与AEC的u32AecSl和u32AecSupIntensity作用一致，可以将整个频段划分来使不同降噪强度作用到不同的频段上。
u32NrIntensity(MI)/ anr_intensity(ANR)	ANR算法的 强度	推荐值：20 说明：该参数表示ANR算法的降噪强度，该值越大，效果越好，但细节损失越多。反之，降噪强度越小，保留越多，但噪声也没法去除干净。(此值也需要微调。) 注意：参数范围为 [0,30]。
u32NrSmoothLevel(MI)/ anr_smooth_level(ANR)	ANR算法的 平滑程度	推荐值：10 说明：该参数表示ANR算法处理的平滑程度，值越大效果越平滑。平滑程度越小在频域上损失的细节越多 注意：参数范围为 [0,10]。
ENrSpeed(MI)/ anr_converge_speed(ANR)	ANR算法的 收敛速度	推荐值：E_MI_AUDIO_NR_SPEED_MID(MI)/1(ANR) 说明：此值表示ANR算法的收敛速度，收敛速度越快的细节越多。反之，收敛速度越慢，参考的时间越长节保留越多，但也会延长降噪起作用的时间。当前的速度已设置为 E_MI_AUDIO_NR_SPEED_HIGH(MI)/2(ANR)，但仍觉得比较慢时，可以打开AEC的 bComfortNoiseEnable(MI)/comfort_noise_enable(让AEC的处理结果主动加入噪声，来辅助ANR的收敛



.....

3.3. EQ (Equalizer)

3.3.1. EQ算法的简单介绍

EQ即均衡器，该算法用于提高或降低某些频段的能量。

3.3.2. EQ算法的参数说明

下面为MI API开给应用设定EQ参数的数据结构：

```
typedef struct MI_AUDIO_EqConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_S16    s16EqGainDb[129];
}MI_AUDIO_EqConfig_t;
```

下面为EQ算法开给应用设定EQ参数的数据结构(仅说明与效果相关的部分)：

```
typedef struct{

    unsigned int eq_enable;

    unsigned int user_mode;

    short eq_gain_db[_EQ_BAND_NUM];

}AudioEqConfig;
```

参数	作用	限制/注意事项/详细说明
eMode(MI)/ user_mode(EQ)	EQ算法的模式	说明：该值表示EQ算法的模式 E_MI_AUDIO_ALGORITHM_MODE_DEFAULT(MI)/0(EQ)：默认模式，完全不吃应用下的参数，全频段放大。 E_MI_AUDIO_ALGORITHM_MODE_USER(MI)/1(EQ)：此模式全部使用应用下的降噪参数。（当用户需要自己调教时使用此模式）
s16EqGainDb(MI)/ eq_gain_db(EQ)	EQ算法的均衡器数组	说明：该数组表示各个频段的增益设置。 该参数把当前采样率对应的最大采样频率分成129份，可单独对某个频段进行能量的调节。假设当前的采样率为16K，则最大的采样频率为8K，每一份对应的频段大小为 $F = 8000 / 129 \approx 62\text{Hz}$ 。此数组元素对应的频段为0~62Hz，62~124Hz，124~192Hz，.....，7938~8000Hz。注意：该参数的范围为[-50,20]dB。当需要调整的增益变化很大时，往往没法在一个频段内完成增益的变化，此时需要将附近相邻的频段也设置为同样的参数，来达到调整该频段的目的。

3.4. AGC (Automatic Gain Control)

3.4.1. AGC算法的简单介绍

AGC即自动增益控制，此算法可以对输入的数据进行增益控制，当信号过小时，会对信号进行增益，当信号过大时，会对信号进行衰减。

3.4.2. AGC算法的参数说明

下面为MI API开给应用设定AGC参数的数据结构：

因为AGC算法更新，数据结构会有所改动。

```
typedef struct AgcGainInfo_s{
    MI_S32    s32GainMax;
    MI_S32    s32GainMin;
    MI_S32    s32GainInit;
}AgcGainInfo_t;
```

```
typedef struct MI_AUDIO_AgcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;
    AgcGainInfo_t stAgcGainInfo;
    MI_U32      u32DropGainMax;
    MI_U32      u32AttackTime;
    MI_U32      u32ReleaseTime;
    MI_S16      s16Compression_ratio_input[5/7];
    MI_S16      s16Compression_ratio_output[5/7];
    MI_S32      s32TargetLevelDb/s32DropGainThreshold
    MI_S32      s32NoiseGateDb;
    MI_U32      u32NoiseGateAttenuationDb;
}MI_AUDIO_AgcConfig_t;
```

下面为AGC算法开给应用设定AGC参数的数据结构(仅说明与效果相关的部分)：

```
typedef struct
{
    int gain_max;

    int gain_min;

    int gain_init;
}AgcGainInfo;

typedef struct
{
    unsigned int agc_enable;

    unsigned int user_mode;

    AgcGainInfo gain_info;

    unsigned int drop_gain_max;

    unsigned int attack_time;

    unsigned int release_time;

    short compression_ratio_input[_AGC_BAND_NUM];
```

```
short compression_ratio_output[_AGC_BAND_NUM];

int drop_gain_threshold;

int noise_gate_db;

unsigned int noise_gate_attenuation_db;

unsigned int gain_step;  //新算法版本支持

}AudioAgcConfig;
```

参数	作用	限制/注意事项/详细说明
eMode(MI)/ user_mode(AGC)	AGC算法的模式	说明：此参数表示AGC算法的场景。 E_MI_AUDIO_ALGORITHM_MO 语音模式，会使用算法中为语音 E_MI_AUDIO_ALGORITHM_MO 用应用下的AGC参数。（当用户 E_MI_AUDIO_ALGORITHM_MO 乐模式，会使用算法中为音乐数
s32GainMax(MI)/ gain_max(AGC)	AGC算法调节增益的最大值(dB)	说明：此值为AGC算法增益可调节 注意：该参数的范围为 [0,60]dB
s32GainMin(MI)/ gain_min(AGC)	AGC算法调节下降的最小值(dB)	说明：此值为AGC算法增益可调节 注意：该参数的范围为 [-20,30]
s32GainInit (MI)/ gain_init(AGC)	AGC算法调节增益的初始值(dB)	说明：此值为AGC算法增益的初 注意：该参数的范围为 [-20,60]
u32DropGainMax(MI)/ drop_gain_max(MI)	AGC算法增益下降的最大值(dB)	说明：此值为AGC算法增益调节 (s32TargetLevelDb/s32DropGain drop_gain_threshold(AGC)时， 出现此情况时，AGC算法会在 0dB~u32DropGainMax(MI)/dro 并不是只要一超过就会衰减u32 drop_gain_max(AGC)dB，且调 stAgcGainInfo(MI)/ gain_info(
参数	作用	限制/该参数的范围说明 号远高于目标值时，增益没法

该值也不宜设置过大，在输入信号突然变小时，信号的增益会瞬间下降太多，导致出现明显的噗噗声。		
u32AttackTime(MI)/ attack_time(AGC)	AGC算法增益下降的时间	说明：此值为AGC算法增益下降0.5dB)，一个时间单位为16m量设小，否则当输入信号高于E s32TargetLevelDb/s32DropGai drop_gain_max(AGC)，没法快工作。注意：该值的范围为 [1,
u32ReleaseTime(MI)/ release_time(AGC)	AGC算法增益上升的时间	说明：此值为AGC算法增益上升0.5dB)，一个时间单位为16m量避免设置太小，否则当经过长的输入信号，则会导致声音爆炸来。此值主要配合曲线来工作。注意：该值的范围为 [1,20]
s16Compression_ratio_input(MI)/ compression_ratio_input(AGC) s16Compression_ratio_output(MI)/ compression_ratio_output(AGC)	AGC算法的增益设置曲线	说明：由s16Compression_ratio_input(AGC) compression_ratio_output(AGC)曲线。 注意：两个数组的元素的参数和compression ratio output的最顶端。注：更改的内容只是增加
s32TargetLevelDb/s32DropGainThreshold(MI)/ drop_gain_threshold(AGC)	AGC算法的增益目标值	说明：AGC算法允许输出的增益行衰减，以防爆音。 注意：此值的参数范围为 [-80,0
s32NoiseGateDb(MI)/ noise_gate_db(AGC)	AGC算法处理的噪声门限值	推荐值：-80 说明：AGC算法处理的噪声门限AGC会认为是噪声，会配合u32 noise_gate_attenuation_db(AG留噪声的话，需要将此值设定到限值之间的这段曲线的斜率设置声。 注意：此参数范围为 [-80,0]dB 当对方AEC实在没法把回声处理noise_gate_db(AGC)和u32Noise noise_gate_attenuation_db(AG时也会对人声有影响（需要权衡限制/注意事项/详细说明
参数	作用	

u32NoiseGateAttenuationDb(MI)/ noise_gate_attenuation_db(AGC)	AGC算法处 理的噪声衰 减值	推荐值：0 说明：AGC算法处理的噪声衰减 s32NoiseGateDb来处理低于门 声进行衰减，防止输出信号听走 说话的开头和结尾被消掉，使声 注意：此参数范围为 [0,100]dB
gain_step	套用增益的 速率	套用增益的速率，以0.5dB为一 套用±0.5dB。此值设定的越高 [1，10]；步长1

Compression ratio input和Compression ratio output是需要根据实际的场景来进行设置（各个产品的器件、结构、适配的机型各不相同，需要具体抓音档来分析），没有默认推荐值。

下面按以

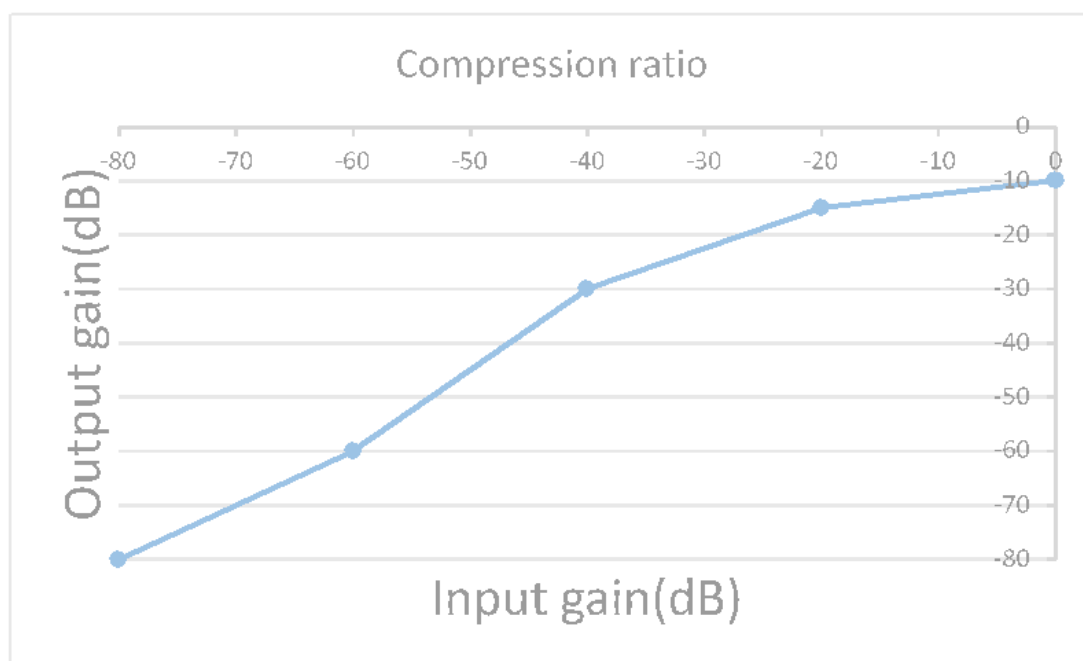
Compression ratio input = {-80,-60,-40,-20,0};

Compression ratio output = {-80,-60,-30,-15,-10};

这组参数进行讲解。如下面的折线图所示，在输入增益为-80~0dB划分为四段斜率：

- 80dB~-60dB范围内保持原来的增益，斜率为1；输出增益为-80~-60dB;
- 60dB~-40dB范围内需要稍微提高增益，斜率为1.5；输出增益为-60~-30dB;
- 40dB~-20dB范围内斜率为1.25；输出增益为-30~-15dB;
- 20dB~0dB范围内斜率为0.25；输出增益为-15~-10dB。

根据所需的曲线转折点对Compression ratio input和Compression ratio output设置，若不需要那么多段曲线，则将数组不需要的部分填0。



(此图仅为了说明曲线的参数及其意义，并无实际参考价值)

4. 设备结构测试

当拿到用户设备时，如何确认设备结构设计的好坏？

主要由以下两点来确认：

- 内部隔离
- 回声损耗(Echo Return Loss (ERL))。

(若这两项没法达到要求，则说明结构设计不太合理，要么建议可以修改结构，要么只能降低对效果的要求，尽量接近)

注：首先必须先确认用户使用的是**HW loop back**还是**SW loop back**，若是使用**HW loopback**必须先对chn1做衰减处理（-6，-3，0dB），保证chn0不会因为chn1，而导致失真。以下所有步骤均在不开启任何音频算法的前提下操作。

4.1. 测试音档介绍

测试音频：

TestFile1_pink_noise [media/TestFile1_pink_noise.wav]，TestFile1为一段噪声，用于辅助确认microphone和speaker的增益。

TestFile2_speech [media/TestFile2_speech.wav]，TestFile2为一段对话语音，主要是外国友人的声音，频段比较全，消除难度相对比TestFile4高，用于调整AEC参数。

TestFile3_sweep [media/TestFile3_sweep.wav]，TestFile3为一段频率逐渐增加的声音，用于扫频，用于检查mic和speaker的频率响应特性。

TestFile4_Chinese_speech [media/TestFile4_Chinese_speech.wav]，TestFile4为一段对话语音，主要是国人的声音，用于调整AEC参数。

4.2. 确定speaker和microphone的增益

确定speaker和microphone的增益需要测量内部隔离和回声损耗。

首先根据用户的需求（mic的收音距离和speaker的音量），通过测量定好microphone和speaker的增益。此步骤为后续调音的最为重要的一步，若用户需要更改这两个增益，所有的算法相关参数都需要重新调整。

4.2.1. 确认speaker的增益

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
2. 用待测设备播放**TestFile1** [media/TestFile1_pink_noise.wav]，调整speaker的增益，在保证没有失真的前提下，使之达到用户的需求。

4.2.2. 确认microphone的增益

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
2. 将分贝仪和待测设备放置在同一位置，在距离待测设备一米处，放置一台播放设备，用播放设备播放**TestFile1** [media/TestFile1_pink_noise.wav]，并调节播放设备的增益，使分贝仪达到70dbA。（没有两台机子请跳转到(4)）
3. 使用待测试设备进行收音，dump A点处的数据（或demo在不打开任何算法的情况下录制的最终数据），调整microphone增益，使RMS达到-25 DBRMS

或以上。（使用[Audacity查看RMS的方法](#) [#jump6]）

4. 确定speaker和microphone的增益后，待测设备使用[确认speaker的增益](#) [#jump1]得到的增益播放[TestFile2](#) [media/TestFile2_speech.wav]，同时使用调整好的microphone增益录制音档，确认microphone录制的音档有没有失真。若有失真请继续调整speaker或microphone的增益（减小speaker的增益或减小microphone的增益）。但有时候会因为结构、speaker、功放等因素，无论你怎么调整录回来的音档都会有某些频段存在失真，此时只能从音档上分析是什么原因造成的，尽量去调整，找到原因。
 5. 若结构设计不合理或者microphone选型灵敏度太低没法达到-25 DBRMS，则在开启AEC后microphone录回来的回音没有爆音的情况下，将microphone的增益调整到足够大。
-

4.3. 内部隔离

内部隔离越大越好，至少需要确保大于6dB。以下为测量内部隔离的步骤：

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
 2. 用待测设备自录自播测试音档[TestFile1](#) [media/TestFile1_pink_noise.wav]（dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的数
据），查看平均RMS振幅，记为AVGRMS1。
 3. 用黏土堵住机壳上的microphone孔，待测设备自录自播测试音档[TestFile1](#) [media/TestFile1_pink_noise.wav]（dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的数
据），查看平均RMS振幅，记为AVGRMS2。
 4. 内部隔离为AVGRMS1 - AVGRMS2。
-

4.4. 回音损耗

回音的损耗同样也是越大越好，至少需要确保这个值为一个正值。以下为测量回音损耗的步骤：

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
2. 用工具查看测试音档[TestFile1](#) [media/TestFile1_pink_noise.wav]的平均RMS振幅，记为AVGRMS1。

3. 用待测设备自录自播测试音档TestFile1 [media/TestFile1_pink_noise.wav] (dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的数
据)，查看平均RMS振幅，记为AVGRMS2。
 4. $ERL = AVGRMS1 - AVGRMS2$ 。
 5. 将microphone和speaker的增益设置为0dB (注意speaker的增益包含功放的
增益，若功放已有做放大，AO的gain则设置成衰减使整体接近0dB)，用测
试音档TestFile3 [media/TestFile3_sweep.wav]代替TestFile1
[media/TestFile1_pink_noise.wav]，自播自录，dump下音档 (dumpA点处的
数据，或者使用demo在关闭所有算法的情况下录制的数)。
 6. 确认该音档在任何频段都不能有失真。
-

4.5. 频响特性

如何初步确定设备的频响特性的好坏？如何分析导致频响特性差的原因？

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
2. 将mic和speaker的增益都设置为0dB (注意speaker的增益包含功放的增益，
若功放已有做放大，AO的gain则设置成衰减使整体接近0dB)，自播自录扫
频音档TestFile3 [media/TestFile3_sweep.wav]，查看收音的音档。

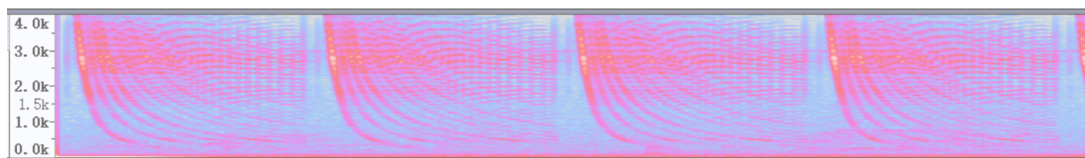
若发现收音音档的频响特性比较差时，可以按以下步骤来确认问题点：

1. 先把机壳拆开，确认mic和speaker是否有固定牢固，重复扫频操作，若频响
特性变好，则确定为mic和speaker没有固定牢的原因。
2. 使用高保真音响播放测试音档TestFile3 [media/TestFile3_sweep.wav]，用标
准麦克风和设备的mic同时拾音，对比两者的频响特性，确认是否为
microphone的频响特性问题 (因mic的集成度较高，一般频响特性没有太大的
问题)。
3. 把speaker拿到机壳外，重复扫频操作，若频响特性变好，则确定为机构问
题。
4. 当把speaker拿到机壳外，使用标准麦克风
[https://zx.ingping.com/c_2/50415.html] (或上面测量频响特性没有太大问
题的mic)，重复扫频动作，若只有小部分频段失真，则可以在确定增益后使
用AO端的EQ来对失真的频段进行修复。

5. 当把speaker拿到机壳外，使用标准麦克风

[https://zx.ingping.com/c_2/50415.html] (或上面测量频响特性没有太大问题的mic)，重复扫频动作。

若基本上是全频段失真如下图。首先尝试下降speaker的增益，若发现已经降到很低了，还是会有全频段的失真，这时需要请硬件工程师帮忙确认功放电路设计是否存在问题，可以修改电路最好。若没法通过修改电路来改善，则一直降speaker的增益，自录自播扫频音档，找到一个最大且不失真的增益（这一步也可通过用示波器来量功放的输出端波形来确定，此方法更加精确，但需要拆卸设备的外壳以及硬件同事的协助），以此增益来作为最大的输出增益。若在此增益上觉得speaker出来的声音大小不满足需求，可以使用AO端的AGC将输出信号拉高，并限制在-3~-1dB以下，来提高声音的大小。



4.6. 结构测试总结

4.6.1. 测试步骤

1. 先关掉用户应用（不启动用户应用）；
2. 在控制台输入dump数据的指令；

```
export MI_AI_DUMP_PATH=XXX(PATH) ;  
  
export MI_AI_DUMP_AEC_ENABLE=1 ;
```

3. 执行命令；

`./prog_audio_all_test_case -t 20 -l -o /tmp/ -d 0 -m 0 -c 1 -s 8000 -v 15`(模拟增益)_3 (数字增益) -b -O -D 0 -V 0 (喇叭增益) -i (扫频音档wav文件的位置)

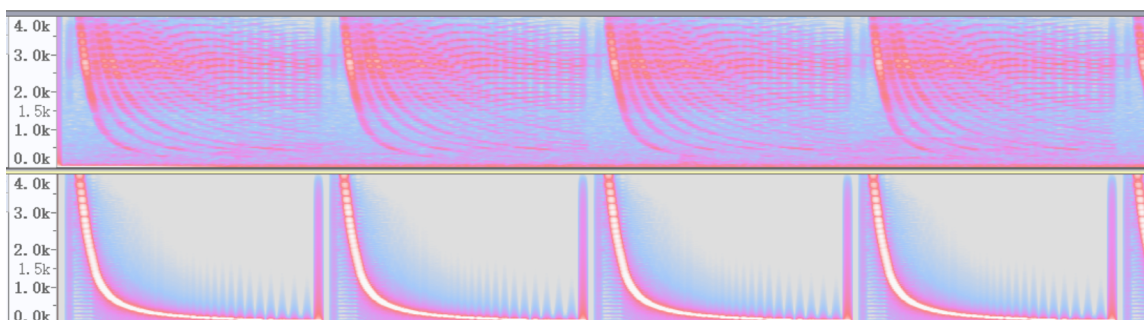
注：具体增益请参考具体机型的设定；

4. 保存XXX(PATH)下生成的音档；
5. 用黏土堵住机器表面的mic收音孔，重复步骤3、4；

6. 对比分析两份音档。

4.6.2. 结果分析

扫频测试本质是一个自播自录的过程，某公司产品扫频结果如下图所示，其中下半部分是原始的扫频音档，上半部分是经由speaker播出mic回收的音档。可以看出两个音档具有很大的差别，说明存在较大的失真。



4.6.3. 解决办法

如果扫频结果有失真，把speaker从壳体拿出来，再次扫频测试来确定是结构问题还是speaker的问题。如果是结构问题，建议用户把speaker和mic堵好，让声音只能从壳体外面进入mic; 如果是speaker的问题，就换个好一点的speaker。speaker明显失真通常是增益太大导致的，对于这种情况可以降低AO的增益。

5. 音效调整

当基础的microphone和speaker增益确认后，且频响特性没有太大的问题，就可以进行音效的调整。

5.1. 确定mic和speaker的增益

如果在步骤[确认microphone的增益](#) [jump2]因为结构设计不合理或者microphone的选型灵敏度太低等原因没法达到-25DBRMS的情况下，请按以下步骤来确定增益，若能达到-25DBRMS，则请沿用前面选定的microphone和speaker增益。

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。

2. 将microphone和speaker的增益（注意speaker的增益包含功放的增益，若功放已有做放大，AO的gain则设置成衰减使整体接近0dB）都设定为0dB。
3. 播放测试音档TestFile3 [media/TestFile3_sweep.wav]，自录自播，查看录制下来的音档（dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的的数据），确认录到的幅值比较小。（此时某些频段有失真不用太过关心，下文有讲解处理的方式）。
4. 播放测试音档TestFile2 [media/TestFile2_speech.wav]，自录自播，将microphone增益在保证不失真的情况下调整到最大（dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的的数据）。
5. 调整speaker到符合用户要求的增益，在调整speaker增益后，microphone增益也要跟随一起调整，大体遵循加一减一，speaker加1dB，microphone相应要降1dB。

在第4步时，可能会因为结构设计不合理，造成mic用很小的增益都会失真，或者说在保证不失真的前提下mic输出的声音振幅很小，请按以下步骤进行尝试调整：

当microphone增益很小就出现了失真，可以根据步骤内部隔离 [#jump3]，确认机构是否合理。若确定为机构问题，但不想修改机构，则需要调小speaker的增益，根据新的speaker增益进行步骤确定mic和speaker的增益 [#jump4]来确定新的microphone增益。或者通过调整speaker端来尝试消除此失真，步骤如下：

1. 寻找一个比较安静的环境，保证环境噪音低于40dbA。
2. 将microphone和speaker的增益都设定为当前增益。
3. 播放测试音档TestFile3 [media/TestFile3_sweep.wav]，自录自播（dumpA点处的数据，或者使用demo在关闭所有算法的情况下录制的的数据）。通过录制的音档查看，寻找到有失真的地方，应用工具的频率分析，找出增益比其他频段要高的频段，通过设置AO的EQ将这些频段做衰减（如何调整AO的EQ table? [#jump5]，此方法不适用于因谐振产生的失真），重复步骤(1)和(2)，若对这些频段调整EQ后还有失真的情况，则只能加大到衰减值继续尝试。

当speaker端使用EQ修正其频率响应，效果仍然没有较大改善时，可以考虑降低microphone的增益，以及打开microphone端的ANR和AGC来对数据进行调整，在AEC处理后的数据中找到消不掉的回声，用ANR和AGC对回声进行衰减。

当进行以上操作得到的microphone数据仍然很小声的时候，只能尝试在拉大microphone增益，让回声爆掉的同时，加大AEC的消回声处理强度。（或建议改

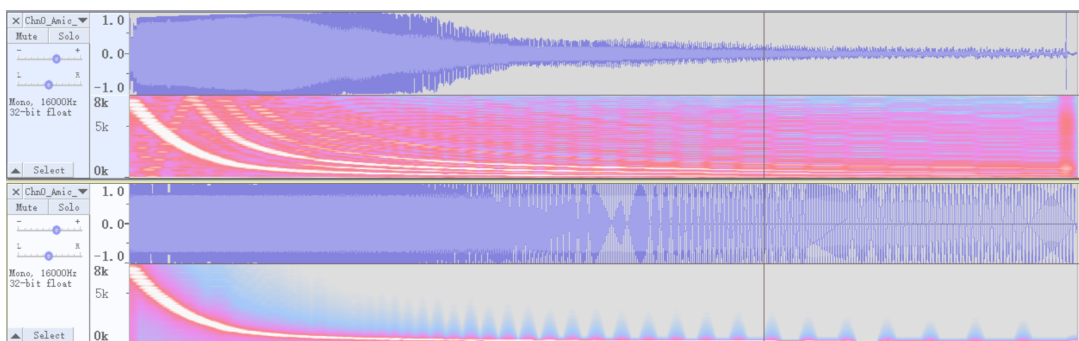
做单向对讲) (若使用HW loopback , 则转为SW loopback)

5.2. 调整AO的EQ参数 , 修正speaker的频响特性

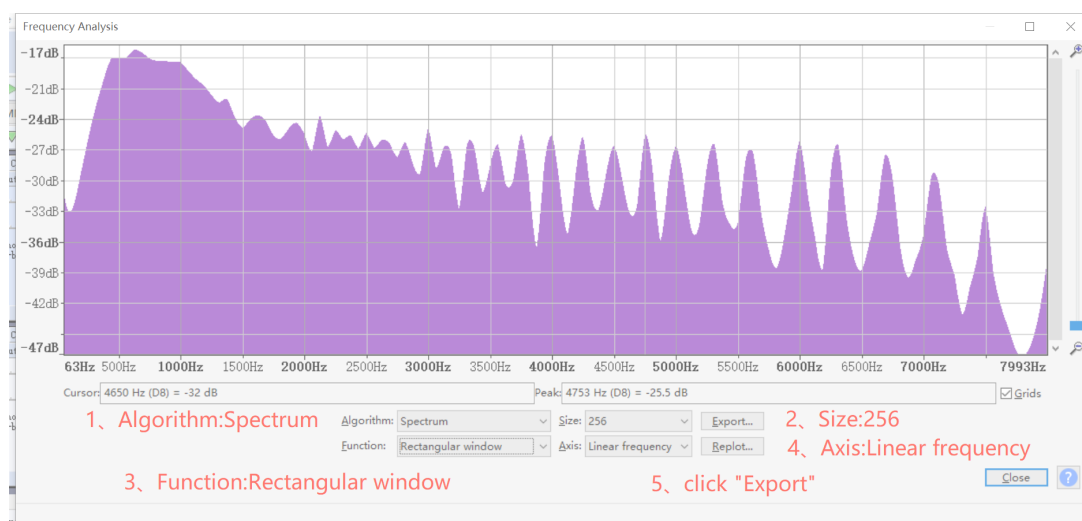
只有当speaker的频响特性非常差 , 或者说有部分固定频率的回声消不掉时 , 才要使用AO的EQ 。

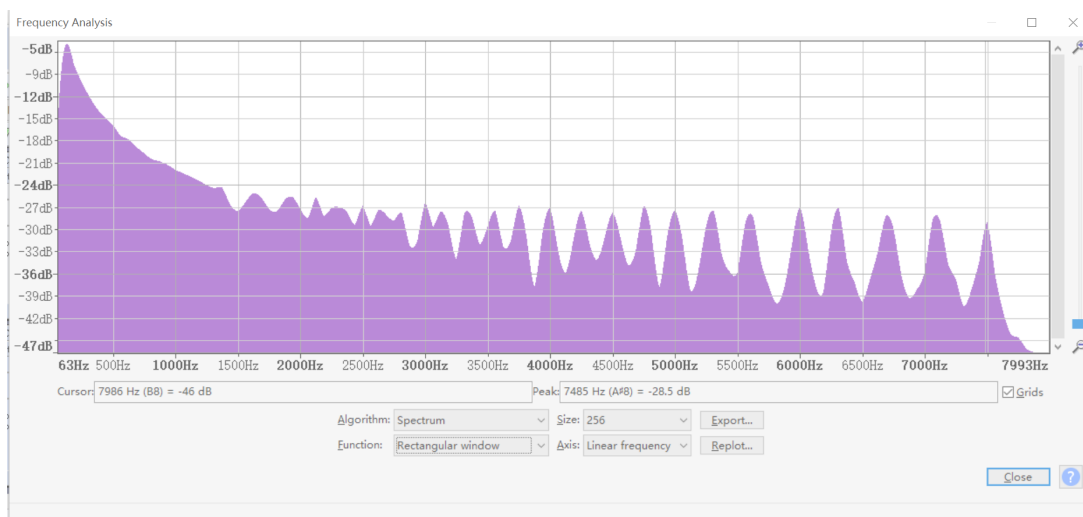
如何得到AO EQ的调整table呢 ? 当只有少部分特定频段会失真时 , 可以直接通过dump到的音档来确认是哪些频段 , 然后在AO EQ的table上直接填写适当的衰减来修正这些失真。但是当录制的音档频响特性较差时 (当失真是由倍频引起时不建议使用) , 可以考虑以下方法来调整AO EQ的table。(此调整是基于当前AI/AO的增益)

1. 在Audacity中导入数据 , 自录自播TestFile3 , 导入录进来的数据 , 导入播放的原始音档 ; 调整音档 , 保证数据在在时间上对齐 。



2. 打开上述音档的频率分析 , 得到频响特性曲线数据 (两个音档分别导出) 。





3. 打开Excel，分别粘贴上面导出的曲线数据。

	A	B	C	D	E
1	Frequency (Hz)	record	playback		
2	62.5	-31.502563	-14.604506		
3	125	-32.745441	-5.111997		
4	187.5	-29.718887	-7.773611		
5	250	-26.178688	-10.127864		
6	312.5	-22.966846	-12.099374		
7	375	-20.357319	-13.740761		
8	437.5	-18.197775	-14.944559		
9	500	-18.042454	-15.907398		
10	562.5	-18.008997	-17.368418		
11	625	-17.28546	-17.81407		
12	687.5	-17.52075	-18.835268		
13	750	-18.104223	-19.707895		
14	812.5	-18.338568	-20.49366		
15	875	-18.370293	-20.778406		
16	937.5	-18.433504	-21.265764		
17	1000	-18.458252	-21.911009		
18	1062.5	-19.316322	-22.400169		
19	1125	-20.114614	-22.868128		
20	1187.5	-20.72348	-23.381828		
21	1250	-21.647701	-23.977119		
22	1312.5	-22.40872	-24.401548		
23	1375	-22.03714	-24.33671		
24	1437.5	-23.786026	-26.560719		
25	1500	-24.815025	-27.463528		
26	1562.5	-24.022215	-26.239586		
27	1625	-23.632862	-25.090282		
28	1687.5	-24.162928	-25.626818		
29	1750	-25.580957	-27.178041		
30	1812.5	-25.885902	-27.485456		
31	1875	-24.900934	-25.9932		
32	1937.5	-24.368271	-25.56464		
33	2000	-25.470238	-27.14216		
34	2062.5	-27.106958	-28.436281		
35	2125	-23.657553	-25.616528		
36	2187.5	-26.71279	-28.124233		
37	2250	-25.092567	-27.041079		
38	2312.5	-25.977554	-26.979763		
39	2375	-25.565149	-27.565805		
40	2437.5	-26.856066	-29.327511		
41	2500	-25.345369	-26.73567		

4. 统计录进来的音档和播放音档在各个频段相差的增益值，以及相差增益值的平均值。

	A	B	C	D
1	62.5	-31.502563	-14.604506	-16.8981
2	125	-32.745441	-5.111997	-27.6334
3	187.5	-29.718887	-7.773611	-21.9453
4	250	-26.178688	-10.127864	-16.0508
5	312.5	-22.966846	-12.099374	-10.8675
6	375	-20.357319	-13.740761	-6.61656
7	437.5	-18.197775	-14.944559	-3.25322
8	500	-18.042454	-15.907398	-2.13506
9	562.5	-18.008997	-17.368418	-0.64058
10	625	-17.28546	-17.81407	0.52861
11	687.5	-17.52075	-18.835268	1.314518
12	750	-18.104223	-19.707895	1.603672
13	812.5	-18.338568	-20.49366	2.155092
14	875	-18.370293	-20.778406	2.408113
15	937.5	-18.433504	-21.265764	2.83226
16	1000	-18.458252	-21.911009	3.452757
17	1062.5	-19.316322	-22.400169	3.083847
18	1125	-20.114614	-22.868128	2.753514
19	1187.5	-20.72348	-23.381828	2.658348
20	1250	-21.647701	-23.977119	2.329418
21	1312.5	-22.40872	-24.401548	1.992828
22	1375	-22.03714	-24.33671	2.29957
23	1437.5	-23.786026	-26.560719	2.774693
24	1500	-24.815025	-27.463528	2.648503

此时得到相差增益值的平均值为0.142025dB。

5. 用各个相差增益值减去平均值。

	A	B	C	D	E
1	62.5	-31.502563	-14.604506	-16.8981	-17.0401
2	125	-32.745441	-5.111997	-27.6334	-27.7755
3	187.5	-29.718887	-7.773611	-21.9453	-22.0873
4	250	-26.178688	-10.127864	-16.0508	-16.1928
5	312.5	-22.966846	-12.099374	-10.8675	-11.0095
6	375	-20.357319	-13.740761	-6.61656	-6.75858
7	437.5	-18.197775	-14.944559	-3.25322	-3.39524
8	500	-18.042454	-15.907398	-2.13506	-2.27708
9	562.5	-18.008997	-17.368418	-0.64058	-0.7826
10	625	-17.28546	-17.81407	0.52861	0.386585
11	687.5	-17.52075	-18.835268	1.314518	1.172493
12	750	-18.104223	-19.707895	1.603672	1.461647
13	812.5	-18.338568	-20.49366	2.155092	2.013067
14	875	-18.370293	-20.778406	2.408113	2.266088
15	937.5	-18.433504	-21.265764	2.83226	2.690235
16	1000	-18.458252	-21.911009	3.452757	3.310732
17	1062.5	-19.316322	-22.400169	3.083847	2.941822
18	1125	-20.114614	-22.868128	2.753514	2.611489
19	1187.5	-20.72348	-23.381828	2.658348	2.516323
20	1250	-21.647701	-23.977119	2.329418	2.187393
21	1312.5	-22.40872	-24.401548	1.992828	1.850803
22	1375	-22.03714	-24.33671	2.29957	2.157545
23	1437.5	-23.786026	-26.560719	2.774693	2.632668
24	1500	-24.815025	-27.463528	2.648503	2.506478

6. 排除掉太大的值后，转换为整数，填入到AO EQ的table中。

如图的F列即为填入AO EQ的数据。

	A	B	C	D	E	F
1	62.5	-31.502563	-14.604506	-16.8981	-17.0401	-18
2	125	-32.745441	-5.111997	-27.6334	-27.7755	-28
3	187.5	-29.718887	-7.773611	-21.9453	-22.0873	-23
4	250	-26.178688	-10.127864	-16.0508	-16.1928	-17
5	312.5	-22.966846	-12.099374	-10.8675	-11.0095	-12
6	375	-20.357319	-13.740761	-6.61656	-6.75858	-7
7	437.5	-18.197775	-14.944559	-3.25322	-3.39524	-4
8	500	-18.042454	-15.907398	-2.13506	-2.27708	-3
9	562.5	-18.008997	-17.368418	-0.64058	-0.7826	-1
10	625	-17.28546	-17.81407	0.52861	0.386585	0
11	687.5	-17.52075	-18.835268	1.314518	1.172493	1
12	750	-18.104223	-19.707895	1.603672	1.461647	1
13	812.5	-18.338568	-20.49366	2.155092	2.013067	2
14	875	-18.370293	-20.778406	2.408113	2.266088	2
15	937.5	-18.433504	-21.265764	2.83226	2.690235	2
16	1000	-18.458252	-21.911009	3.452757	3.310732	3
17	1062.5	-19.316322	-22.400169	3.083847	2.941822	2
18	1125	-20.114614	-22.868128	2.753514	2.611489	2
19	1187.5	-20.72348	-23.381828	2.658348	2.516323	2
20	1250	-21.647701	-23.977119	2.329418	2.187393	2
21	1312.5	-22.40872	-24.401548	1.992828	1.850803	1
22	1375	-22.03714	-24.33671	2.29957	2.157545	2
23	1437.5	-23.786026	-26.560719	2.774693	2.632668	2
24	1500	-24.815025	-27.463528	2.648503	2.506478	2

5.3. AEC频段和强度调整

当我们确定好speaker和microphone的增益后，就需要对AEC的频段和强度进行调整。

1. 将AEC的频段划分和强度设置成以下参数，并进行单向对讲（可用自录自播TestFile2进行模拟）。

```
u32AecSupfreq[6] = {20,40,60,80,100,120}; // MI
```

```
u32AecSupIntensity[7] = {4,4,4,4,4,4,4}; // MI
```

```
suppression_mode_freq[6] = {20,40,60,80,100,120}; // AEC
```

```
suppression_mode_intensity[7] = {4,4,4,4,4,4,4}; // AEC
```

2. dump得到AEC处理后的音档，查看音档是否有消不干净的回声，有则找到回声对应的频段，对上面的频段划分进行调整，将回声的频率范围调整成一个频段，设置到u32AecSupfreq(MI)/ suppression_mode_freq(AEC)数组中，同

时调整u32AecSupIntensity(MI)/ suppression_mode_intensity(AEC)数组，增强对此频段的消回声强度。

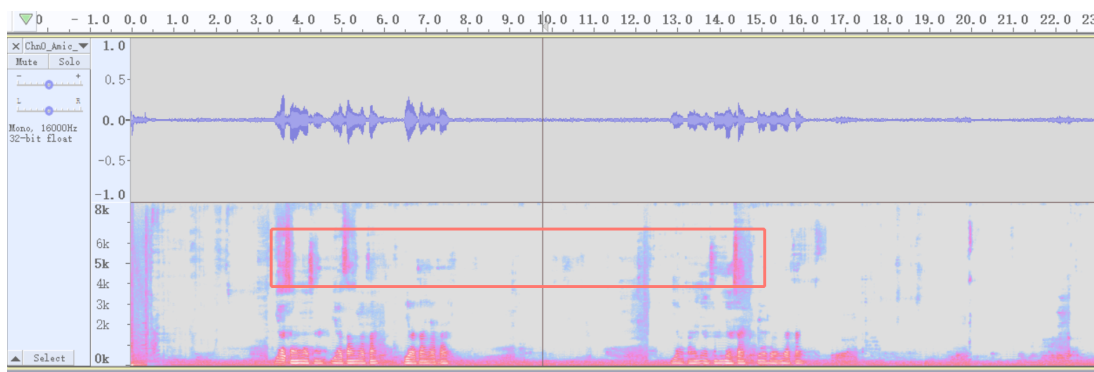
3. 当从dump得到的AEC处理后的音档听到有数据被误砍了，可以将AecAiln、AecAoIn、AecOut三个音档放到Adobe audition里面对比，看被误砍到的频段是否对应的时间点AecAoIn中是否有较大的能量，若没有则调整u32AecSupIntensity数组，降低该频段的AEC消除强度。

举例：如下图所示，通过音档AecOut（听音档、看频谱）我们发现在4K7K的频段还有回声未消除干净，我们可以将4K7K划分为多个AEC消除的段，调整该范围的AEC消除强度。 $4000/62.5=64$ ， $7000/62.5=112$ ，故我们可以将AEC的参数调整为：

```
u32AecSupfreq[6] = {20,40,64,90,112,120}; // MI u32AecSupIntensity[7] = {4,4,4,6,6,4,4};
```

```
u32AecSupfreq[6] = {20,40,64,90,112,120}; // AEC u32AecSupIntensity[7] = {4,4,4,6,6,4,4};
```

（AEC的消除强度则需要根据实际情况来尝试）



注：当有某些固定的频段的回声一直消不掉时，可以使用mic端的EQ对这些频段来做衰减，来消掉这些回声。

5.4. ANR参数调整

当我们录进来的声音因为环境或硬件问题存在持续的噪声时，则可以使用ANR来进行降噪，但是ANR没法处理那些突然出现的噪声。测试ANR前先不要说话，让ANR收一段音作为参考。

1. 将ANR的默认参数按以下设置，在Vqe设置中只使能AEC和ANR，抓取APC算法前后的音档。

```
eMode = E_MI_AUDIO_ALGORITHM_MODE_MUSIC; // MI
u32NrIntensity = 20;
u32NrSmoothLevel = 10;
eNrSpeed = E_MI_AUDIO_NR_SPEED_MID;
```

或

```
eMode = E_MI_AUDIO_ALGORITHM_MODE_MUSIC; // MI
u32NrIntensityBand[6] = {20, 40, 60, 80, 100, 120};
u32NrIntensity[7] = {20, 20, 20, 20, 20, 20, 20};
u32NrSmoothLevel = 10;
eNrSpeed = E_MI_AUDIO_NR_SPEED_MID;

user_mode= 2; // ANR
anr_intensity_band [6] = {20, 40, 60, 80, 100, 120};
anr_intensity [7] = {20, 20, 20, 20, 20, 20, 20};
anr_smooth_level= 10;
anr_converge_speed= 1;
```

2. 查看APC算法处理后的音档。
3. 若觉得ANR算法收敛的时间过长（即音档中噪声从开始到稳定下来的时间），则增大eNrSpeed。当把eNrSpeed设置到最大时，收敛时间仍然没法接受，可将AEC的bComfortNoiseEnable打开，让AEC加入噪声辅助ANR收敛。
4. 若ANR算法收敛后，仍觉得噪声没法接受，则增大u32NrIntensity。
5. 当遇到环境中只有某些频段噪声比较大，可以使用带频率划分的NR参数（u32NrIntensityBand），单独针对某个频段进行降噪强度的调整（需当前SDK版本支持）。

5.5. AGC参数调整

AGC的参数设置完全依赖AEC，ANR，EQ等算法的处理效果来进行调整，使用音频分析工具来看录制下来的音档，根据用户的需求对各个增益范围进行处理。

注：使用AGC后，输出信号没有任何变化时，请检查 Gain info、release time、attack time、曲线斜率、noise gate是否设置正确。若开启AGC后：

1. 使用以下参数作为AGC起始参数：

```

eMode = 1; // MI
s32GainMax = 30;
s32GainMin = 0;
s32GainInit = 0;
u32DropGainMax = 36;
u32AttackTime = 1;
u32ReleaseTime = 3;
s16Compression_ratio_input[_AGC_BAND_NUM] =
{-80, -60, -40, -20, 0};
s16Compression_ratio_output[_AGC_BAND_NUM] =
{-80, -40, -20, -10, -5};
s32DropGainThreshold = -5;
s32NoiseGateDb = -80;
u32NoiseGateAttenuationDb = 0;

compression_ratio_input[AGC_CR_NUM] = {-80, -60, -40, -20, 0};
compression_ratio_output[AGC_CR_NUM] = {-80, -40, -20, -10, -5};
user_mode = 1;
gain_max = 30;
gain_min = 0;
gain_init = 0;
drop_gain_max = 36;
attack_time = 1;
release_time = 3;
noise_gate_db = -80;
noise_gate_attenuation_db = 0;
drop_gain_threshold = -5;

```

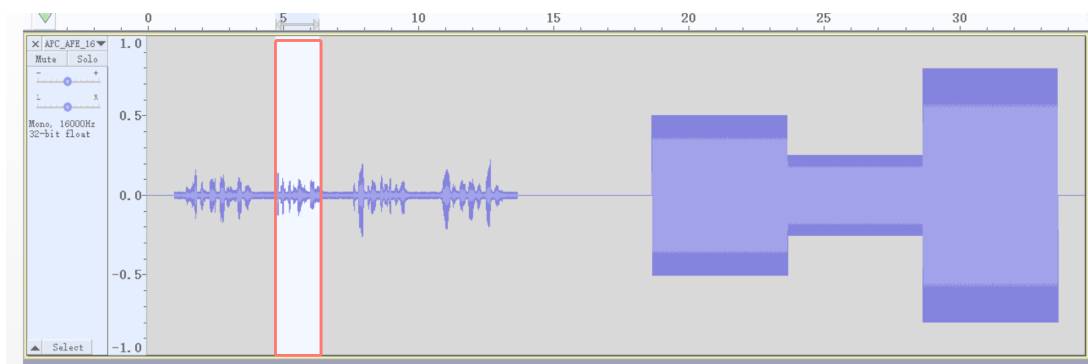
2. 使能Vqe的AEC, ANR, EQ(若有使用EQ)，dump出APC算法处理后的数据并查看，分析出要增强的声音和要抑制的声音分别所在的dB范围（参考[查看增益](#) [#jump6]），根据应用需求调整曲线的斜率，即想要增强的部分，增加曲线的斜率，想要抑制的部分，减小曲线的斜率，初步调整曲线。
3. 将Vqe的AEC,ANR,(EQ，若有使用EQ)，AGC使能，dump出APC算法处理后的数据查看，是否符合预期。
4. 若数据存在大量削顶时，可加大
u32DropGainMax(MI)/drop_gain_max(AGC)、减小对应曲线的斜率、减小
s32DropGainThreshold(MI)/s32DropGainThreshold(AGC)、以及增加
u32ReleaseTime(MI)/release_time(AGC)等方式来缓解。
5. 若数据的增益没有达到要求，可增加对应曲线的斜率、减小
u32ReleaseTime(MI)/release_time(AGC)、增大
s32GainMax(MI)/gain_max(AGC)等方式解决。

6. 若数据没有任何变化，请检查参数是否设置正确。

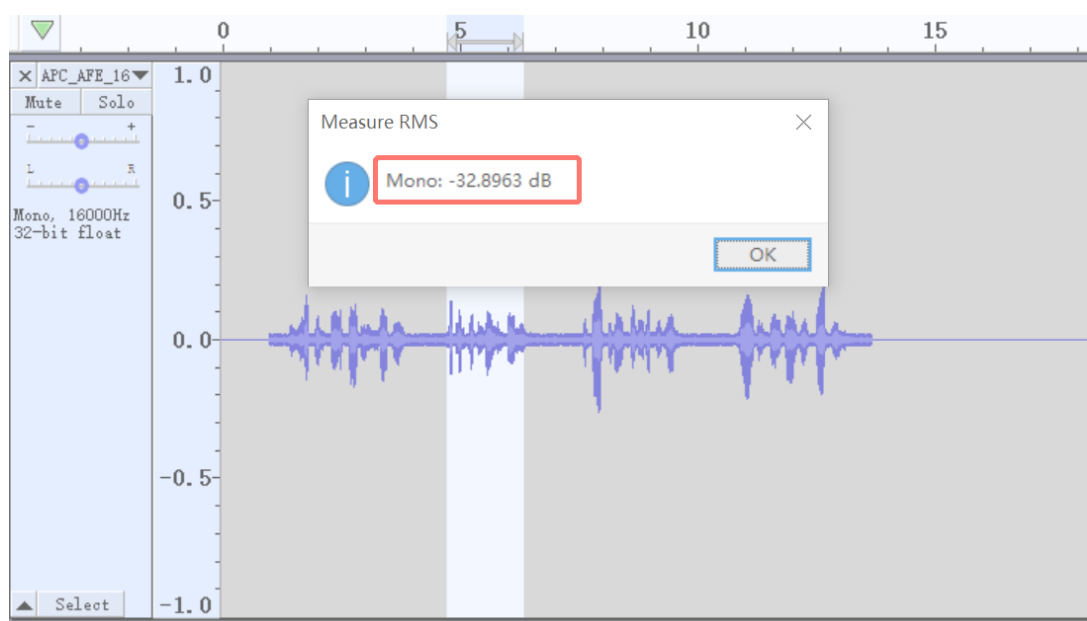
举例：

下面图例说明怎么调整AGC参数将语音信号的增益拉起来。

当我们需要拉起某个能量范围的信号时，需要先从音档中选出需要调整的数据。



查看这段数据的能量大小。



假设我们想把这个信号拉起来，可以通过调整AGC的曲线的斜率来达到这个目的。由上图得到的平均振幅，可以知道该信号在曲线的-40dB~-20dB的段上，那最简单的做法就是将该端的斜率拉起来。这样相当于-40dB~-20dB的信号拉到-20dB到-5dB。

```
s16Compression_ratio_input(MI)/compression_ratio_input[(AGC)
[AGC_CR_NUM] = {-80, -60, -40, -20, 0};

s16Compression_ratio_output(MI)/compression_ratio_output[(AGC)
[AGC_CR_NUM] = {-80, -40, -20, -5, -5};
```

6. 音效调整结构设计建议以及硬件选型

6.1. Speaker和microphone的选型建议

1. microphone最好选择灵敏度在-35dB以下的，但不建议太小，以免达不到收音距离的要求。
 2. 必须选择带有外壳的speaker。
-

6.2. 结构设计建议

1. Microphone和speaker之间的距离最好在4~10cm，距离越远效果越好。
 2. 禁止在结构中使用裸露的speaker，speaker外壳前后之间的紧密连接有助于降低声音从后腔到扬声器前部的传播。
 3. Speaker的外壳必须牢牢地固定在空腔内，可以避免扬声器产生咔哒咔哒的杂音。
 4. 当将speaker安装到结构内时，可使用高密度泡沫橡胶（防震，隔音）来使speaker更加紧密牢固。
 5. 结构上Speaker出声位置的开孔至少要达到speaker面积的20%以上，且开孔应距离speaker 1~2mm。
 6. 可在microphone后面加上泡沫橡胶，来减少speaker和microphone的直接耦合。更进一步可以将麦克风和泡沫橡胶装到一个独立的外壳内。
 7. Microphone必须对准机壳上的开孔。
-

7. 基本调试流程

以下记录一次简单的调试过程，介绍大概调整的流程，以下参数仅适用于某款机型，不具有参考意义。

1. 确定AO的增益

根据[确认speaker的增益](#) [#jump1]章节介绍，在设备上使用demo播放TestFile1和TestFile2，根据用户的要求选择一个足够大声的增益。（因此机型用户没有给出具体要求speaker要多响，这里自行先选择一个合适的增益）。此处选择0dB作为AO的增益。

2. 确定AI的增益

用上述选定的初始增益，跟据[确认microphone的增益](#) [#jump2]章节介绍，将设备放置和分贝仪一同放置在桌上，在距离设备一米处，放置一个可播放音频的设备，使用播放设备播放TestFile1，调整播放设备的音量，使分贝仪能达到70dBA。此时逐步调节microphone的增益，使收进来的声音足够大声，最好收进来的声音能到-25DBRMS。由于手上的机型mic的灵敏度比较低，故mic增益设定到54dB才能勉强达到接近-25DBMS。

3. 测试结构的内部隔音效果

用上述选定的初始增益，按照[内部隔离](#) [#jump3]章节介绍的步骤测量，内部的隔音效果如何。自录自播TestFile1，得到平均RMS振幅，记为AVGRMS1，堵住mic孔，重复上面的操作，记为AVGRMS2，内部隔音= AVGRMS1 - AVGRMS2。

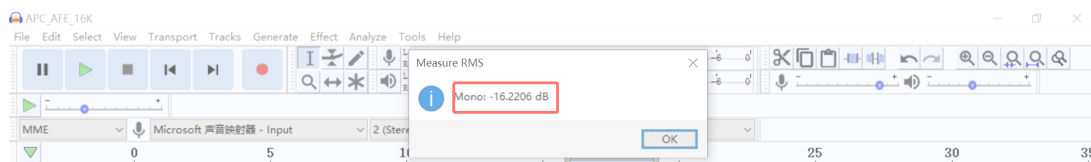


图7-1 内部隔音效果AVGRMS1

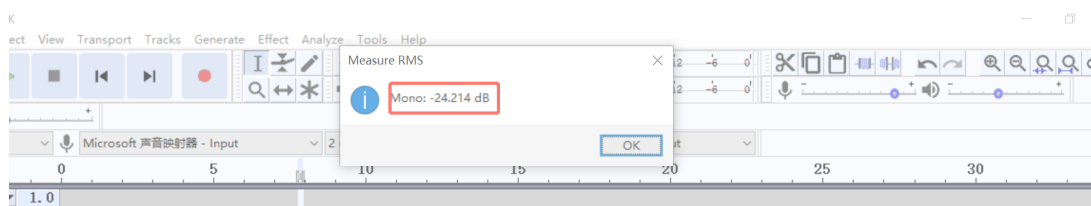


图7-2 内部隔音效果AVGRMS2

$AVGRMS1 - AVGRMS2 = -16.2206\text{db} - (-24.214\text{db}) > 6$ ，由此可见内部的隔音效果满足要求。

4. 测试结构的回声损耗

按照[回音损耗](#) [#jump7]的步骤测量，回声损耗的效果如何。TestFile1音档本身的平均RMS振幅记为AVGRMS1，自录自播TestFile1，得到平均RMS振幅，

记为AVGRMS2。下图分别为AVGRMS1和AVGRMS2。由此可见该机构对回声几乎没有损耗，设计较差。

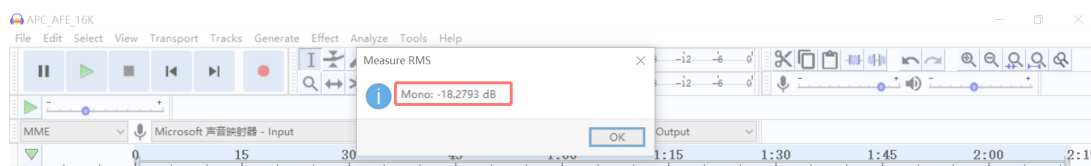


图7-3 回声损耗AVGRMS1

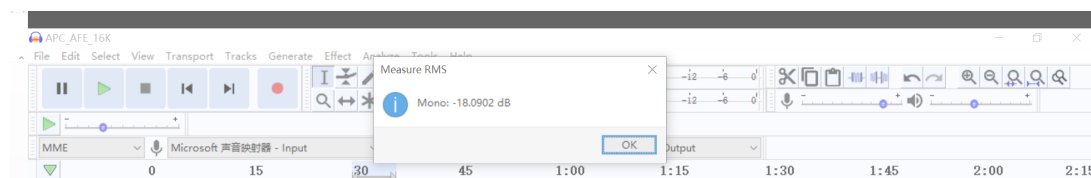
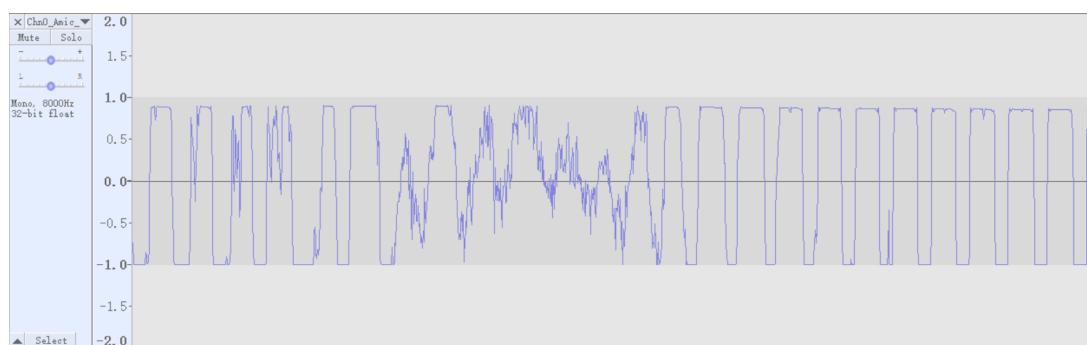


图7-4 回声损耗AVGRMS2

5. 调整AI和AO的增益

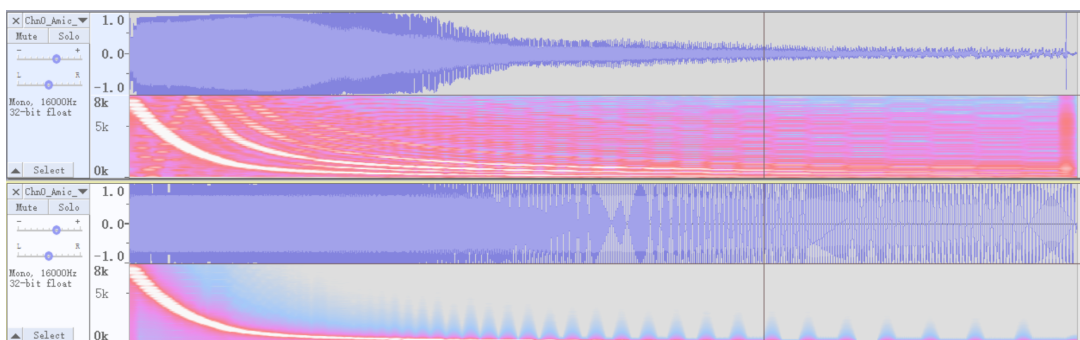
微调AI和AO的增益，使自录自播TestFile2，尽量不会出现失真。若有出现失真，这可以通过减小microphone和speaker的增益来调整到不失真（speaker播出来就失真这种情况除外）。



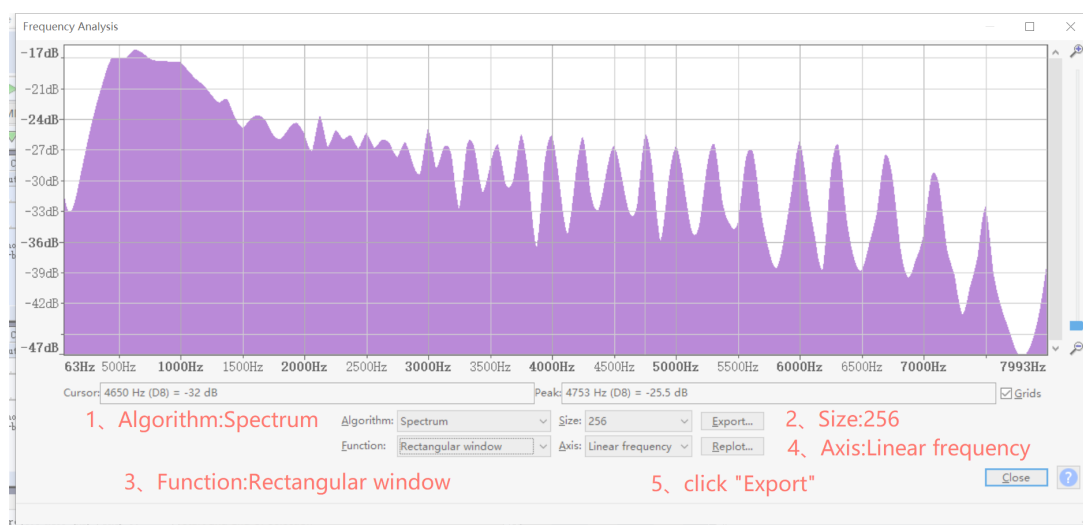
由上图看出部分波形已经削顶了，我们需要对microphone和speaker的增益进行调整。用分贝计在距离一米处进行测试，发现speaker播放TestFile2时，已经达到约75dBA，这个值已经是比较大的了，对speaker增益进行调整，调整到分贝计显示65dBA左右。经过调整后确定microphone和speaker增益为48dB和-9dB。

6. 对speaker进行扫频，修正speaker的EQ

首先我们在上面确定的增益基础上，自录自播扫频音档TestFile3。导入录进来的数据，导入播放的原始音档；调整音档，保证在时间上对齐。



点击频率分析，设置相应的参数得到该音档的频响特性。



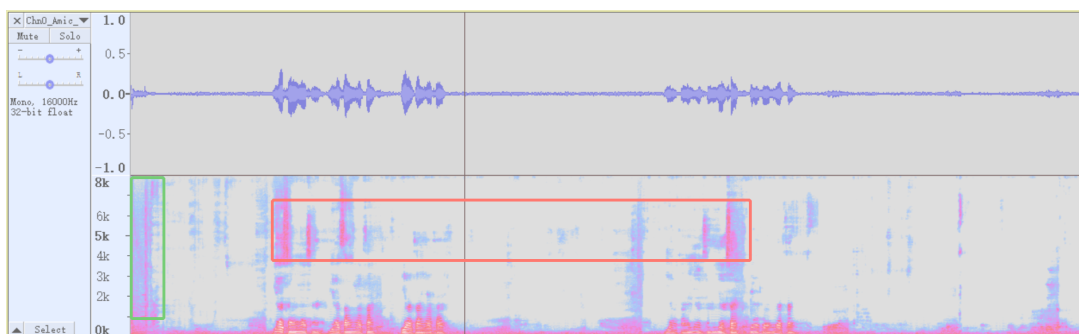
得到曲线的信息，复制到Excel中，计算出AO的EQ table。

	A	B	C	D	E	F
1	62.5	-31.502563	-14.604506	-16.8981	-17.0401	-18
2	125	-32.745441	-5.111997	-27.6334	-27.7755	-28
3	187.5	-29.718887	-7.773611	-21.9453	-22.0873	-23
4	250	-26.178688	-10.127864	-16.0508	-16.1928	-17
5	312.5	-22.966846	-12.099374	-10.8675	-11.0095	-12
6	375	-20.357319	-13.740761	-6.61656	-6.75858	-7
7	437.5	-18.197775	-14.944559	-3.25322	-3.39524	-4
8	500	-18.042454	-15.907398	-2.13506	-2.27708	-3
9	562.5	-18.008997	-17.368418	-0.64058	-0.7826	-1
10	625	-17.28546	-17.81407	0.52861	0.386585	0
11	687.5	-17.52075	-18.835268	1.314518	1.172493	1
12	750	-18.104223	-19.707895	1.603672	1.461647	1
13	812.5	-18.338568	-20.49366	2.155092	2.013067	2
14	875	-18.370293	-20.778406	2.408113	2.266088	2
15	937.5	-18.433504	-21.265764	2.83226	2.690235	2
16	1000	-18.458252	-21.911009	3.452757	3.310732	3
17	1062.5	-19.316322	-22.400169	3.083847	2.941822	2
18	1125	-20.114614	-22.868128	2.753514	2.611489	2
19	1187.5	-20.72348	-23.381828	2.658348	2.516323	2
20	1250	-21.647701	-23.977119	2.329418	2.187393	2
21	1312.5	-22.40872	-24.401548	1.992828	1.850803	1
22	1375	-22.03714	-24.33671	2.29957	2.157545	2
23	1437.5	-23.786026	-26.560719	2.774693	2.632668	2
24	1500	-24.815025	-27.463528	2.648503	2.506478	2

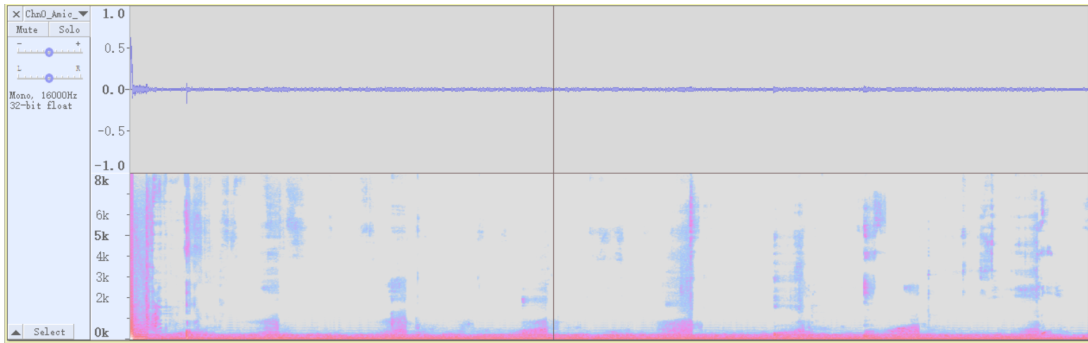
填入到AO的EQ table中。

7. 调整AEC参数

首先我们填入AEC的初始化参数，设置环境变量，先进行single talk测试，自录自播TestFile2，录出AEC的相关音档。由AecOut的音档可以看出消不掉基本只有红色标记对应频段，绿色标记处为AEC算法还没收敛，不用管，接着调整AEC的参数，对这两个频段的AEC强度。



调整参数后，录制的AecOut音档如下：



在调试过程中，如果怎么调AEC的强度回声都没法完全消掉，即使将AEC的强度开到了最大，仍然没法消掉对应部分的回声。只能将参数调整至 microphone收音仍然比较大，且回声较小的情况，后面利用EQ来尝试消掉。

8. 调整AI NR参数

此处基本上我们将默认参数填入即可，当发现因电路和测试环境等因素，发现降噪效果不理想时，再做微调。

```
eMode : E_MI_AUDIO_ALGORITHM_MODE_MUSIC  
  
u32NrIntensity : 20  
  
u32NrSmoothLevel : 10  
  
eNrSpeed : E_MI_AUDIO_NR_SPEED_MID
```

9. 调整AI EQ参数

除了在某些频段的噪声消不掉，其他情况不建议直接使用AI 的EQ来进行调整增益的处理。

10. 调整AI AGC参数

跟AEC一样，我们先填入起始参数再进行微调。起始参数如下：

```
compression_ratio_input[AGC_CR_NUM] = {-80,-60,-40,-20,0};  
compression_ratio_output[AGC_CR_NUM] = {-80,-40,-20,-10,-5};  
user_mode = 1;  
gain_max = 30;  
gain_min = 0;  
gain_init = 0;  
drop_gain_max = 36;  
attack_time = 1;  
release_time = 3;  
noise_gate_db = -60;
```

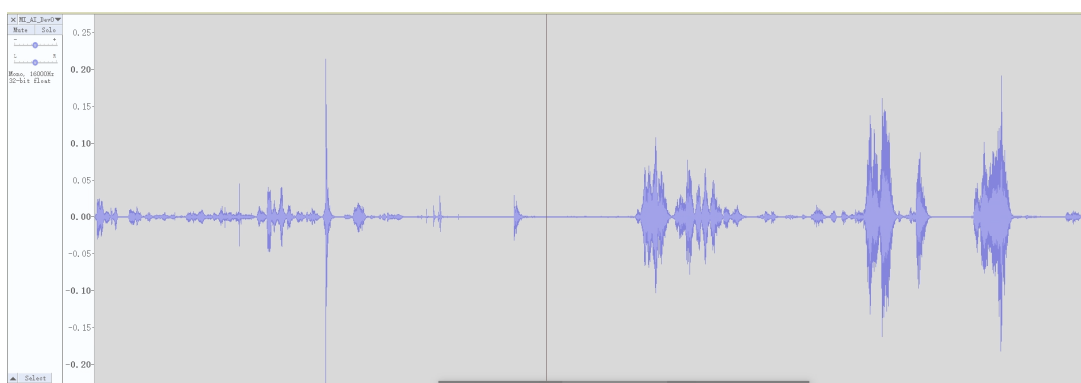
```
noise_gate_attenuation_db = 0;  
target_level_db = -5;
```

接着我们分析AEC处理后的音档。

我们选取几段想要调整的范围，查看他们的平均DBRMS。



再更改默认参数曲线中的output参数，调整到想到的dB值。调整如下：



以上突然飙高的地方均为喇叭失真导致，突然出现且没法消除的回声被AGC拉高了，这个除了改喇叭改结构没有其他的办法。

8. 常见案例

1. 功放电路失真

功放电路失真的案例，主要在于如何来确定问题点在功放电路上，以及确认问题后相应的解决方法。

当我们拿到用户的机器，来进行初步的扫频分析时（将mic和speaker的增益设定为0dB，自播自录扫频音档），若此时看到的频响特性非常差，几乎所有

频段都有谐振，且能量也比较高。这时就需要怀疑功放电路在当前的状态下已经出现了非常严重的失真。

在确定mic频响特性没有太大问题的前提下，可通过更换喇叭来进行测试。若更换了几个喇叭后，对比音档频响特性均没有较大的改善，这时则需要请硬件同事测量功放电路的输入和输出信号，来进行确定。若speaker增益已经衰减得比较大，功放的输入信号正常幅值较小，但功放的输出信号仍然出现失真，就要请硬件同事确认是否为功放放大倍数太大，或者是功放的供电电压太小无法满足放大的需要，出现削顶。这时若能修改功放电路解决最好。

但在不可抗力因素下，比如用户已量产、硬件修改对用户影响太大或修改难度大等原因，我们只能在不失真的前提下，尽量提高对讲时的信号幅值来弥补声音小的缺陷。

首先请硬件同事帮忙测量在播放扫频音档时，speaker增益最大下到多少可以保证功放电路的输出信号不失真，以此增益作为软件可调整的最大增益值，同时为了使speaker输出的声音足够大，使用AO端的AGC尽量对输出数据拉大，并限制在-3~-1dB以下（留有余量，避免功放失真）。

2. Speaker端源数据爆音处理

在对讲时发现即将播放的数据（即far end传过来的原始数据）会经常爆掉的情况下，可以对传过来的数据进行相应的处理来减缓这种情况，可以对数据用多点平均（即将当前采样点的数值与前面多个点进行平均，至于多少个点做一次平均请按具体情况来调整，爆得越严重要求的点数越多）的做法来将一些爆掉的点拉下来。

计算公式：

$$y = \frac{x(n)}{n} + \frac{x(n-1)}{n} + \frac{x(n-2)}{n} + \dots + \frac{x(0)}{n}$$

相当于做了一次低通滤波，可以有效地消除锯齿波，副作用就是周期短的高频信号会被滤掉。多点平均的做法会把音质变糊，其他情况不建议使用。

3. 对方speaker播出来的语音间，有噪声被拉起后，又被压掉

当播出来的语音底噪忽大忽小时，首先需要先确认是哪一方的原因。我们可以通过dump出AI的VqeIn和VqeOut音档来确定，用adobe audition听音档VqeOut是否有底噪忽大忽小的现象。若VqeOut音档听起来正常，则是对方的原因，需要对方来解。

若VqeOut音档听起来就已经存在忽大忽小的噪声，而VqeIn音档中没有，那基本可以肯定是Vqe的处理过程导致的。该现象可能有以下几个原因：

- a. 语音间隔短，导致NR收敛慢。

解决办法：

加快NR的收敛速度，eNrSpeed = E_MI_AUDIO_NR_SPEED_HIGH。

使能AEC的舒适噪声，bComfortNoiseEnable = TRUE。

- b. EQ调整不当。

若从VqeOut音档中看出噪声是在固定频段，可通过调整相应频段的EQ，来压掉噪声。

若VqeIn音档听起来已经有忽大忽小的噪声，那可以dump出Aec的音档，对比AecAiln和AecOut音档，确认是否是Aec过程产生的还是环境中的声音本来就是这样。若只有AecOut音档有，则基本可以确认是AEC消不干净导致的，查看“噪声”（回声）处在哪个频段，将该频段的AEC消除强度加大到可以消掉此回声。

解决方法：调整AEC的u32AecSupfreq和u32AecSupIntensity来处理。

4. 我方speaker播放有“尾音”。

这种情况主观表现为我方speaker播放出来有尾音。首先需要先dump出AO的VqeIn音档来分析，若VqeIn的数据就已经有“尾音”，则是对方AEC消不干净导致的。我们可以通过以下几个方法来处理：

- a. 调整对方的AEC参数。
- b. 若“尾音”的频段比较固定，我们可以通过调整EQ来砍“尾音”所处的频段。
- c. 若“尾音”频段不固定，则只能分析尾音的能量大小，在“尾音”和正常声音能量相差较大时，可通过AGC来处理（仅适用于对方是固定的机子，不存在我方需要对很多不同机子的情况，且很可能会误伤到正常的语音信号）。我们可以分析尾音的信号能量大小（[查看增益](#) [#jump6]），然后调整ACG的曲线（s16Compression_ratio_input和s16Compression_ratio_output）或噪声门限值（s32NoiseGateDb和u32NoiseGateAttenuationDb）将“尾音”压掉。

5. 单向对讲对方消不干净

我方能听到回声，原因是对方消不干净。这种情况下，我们要先定位是什么原因导致对方AEC消不干净。一般主要是以下几个因素导致对方消不干净：

a. 对方的speaker或mic增益太大

解决方法：降低对方的speaker或mic的增益，降低我方的mic增益

b. 对方结构较差，内部串音严重。

确认方法：

- 用黏土堵住对方的收音孔，在对方处说话，我方是否能听到对方的声音。若能听到，则说明对方的结构气密性较差。
- 用黏土将对方mic的背面完全堵住，在我方说话，看回音是否有非常大的改善，若有非常大的改善，则说明是对方结构的问题。

解决方法：对方调整结构

c. 对方AEC强度调整不恰当

解决方法：对方调整AEC强度

注：若对方传回来的回音，频段固定，且能量也不是特别大时，可以通过AO端的EQ来砍对方消不干净的回音。或频段不固定，但回声和正常语音能量相差较大，可通过AO端的AGC曲线（s16Compression_ratio_input和s16Compression_ratio_output）或噪声门限值（s32NoiseGateDb和u32NoiseGateAttenuationDb）将回声压掉。（同样仅适用于只配对一台设备的情况，且可能误伤正常语音信号）

6. 单向对讲我方消不干净

我方消不干净原因与对方消不干净的原因大同小异，完全可以同样的思路来处理。

a. mic/speaker的增益太大导致声音爆掉。

解决方法：降低我方的mic/speaker的增益，降低对方mic的增益

b. 我方结构较差，内部串音严重。可以通过以下两种方式来确认。一、用黏土堵住我方的收音孔，在我方处说话，对方是否能听到我方的声音。若能听到，则说明我方的结构气密性较差。二、用黏土将我方mic的背面完全堵住，在对方说话，看回音是否有非常大的改善，若有非常大的改善，则说明是我方结构的问题。

解决方法：我方调整结构

c. 我方AEC参数没调好

分析Aec的相关音档AecAiln、AecAoIn、AecOut，看消不干净的回声主要在哪些频段，调整频段参数和消除强度（u32AecSupfreq和

u32AecSupIntensity)

注：若我方AEC处理完仍有回声，频段固定，且能量也不是特别大时，可以通过AI端的EQ来砍消不干净的回音。或频段不固定，但回声和正常语音能量相差较大，可通过AI端的AGC曲线 (s16Compression_ratio_input和s16Compression_ratio_output) 或噪声门限值 (s32NoiseGateDb和u32NoiseGateAttenuationDb) 将回声压掉。(同样仅适用于只配对一台设备的情况，且可能误伤正常语音信号)

7. 双向对讲，对方听到的声音忽大忽小

双向对讲时，对方听到的声音忽大忽小。首先要先确定是哪一方的问题，可以通过dump我方AI的AEC、VQE音档，通过分析VqeOut来确认是否是我方引起的问题，若VqeOut音档没有问题，则是对方的原因，需要对方解。若是我方导致的问题则可以通过以下几个因素来进行排查：

- a. 对方回过来的回声太大，导致我方AEC把mic的信号消得比较多。

解决方法：

调整我方的AEC参数，通过分析AEC音档AecAiln、AecAoIn、AecOut，判断是哪些频段AEC开得太强了，重新调整AEC的频段和消除强度 (u32AecSupfreq和u32AecSupIntensity)，或降低speaker的增益。

- b. 我方mic端使能了AGC，但AGC参数没调好。若关掉AGC，则不会忽大忽小，则说明是AGC参数没调好。

解决方法：

结合音档调整AGC参数，主要需要关注曲线 (s16Compression_ratio_input和s16Compression_ratio_output)、噪声门限值 (s32NoiseGateDb和u32NoiseGateAttenuationDb) 以及 u32DropGainMax等设定。

8. 双向对讲，我方听到的声音忽大忽小

双向对讲，我方听到的声音忽大忽小，首先要先确定是哪一方的问题，可以通过dump我方AO的VqeIn和音档VqeOut来确认是否是我方引起的问题，若VqeIn音档就已经忽大忽小，则是对方处理的问题。若VqeIn音档没有忽大忽小，而VqeOut有忽大忽小的现象，则可以通过以下方向来排查：

- a. 我方speaker端使能了AGC，但AGC参数没调好。若关掉AGC，则不会忽大忽小，则说明是AGC参数没调好。

解决方法：

结合音档调整AGC参数，主要需要关注曲线
(s16Compression_ratio_input和s16Compression_ratio_output)、噪声
门限值 (s32NoiseGateDb和u32NoiseGateAttenuationDb) 以及
u32DropGainMax等设定。

b. 结构、喇叭等原因。

比如喇叭没装稳，喇叭某些频段打不出来等。

9. 双方同时说话漏字

双方同时说话漏字，主要由以下原因造成（双方都有可能）：

- a. 漏字方AEC强度开得太强
- b. 其中一方回声消不干净，导致另一方AEC误砍语音信号
- c. mic/speaker增益开得太大，影响AEC处理结果
- d. EQ/AGC等算法参数设置不合理

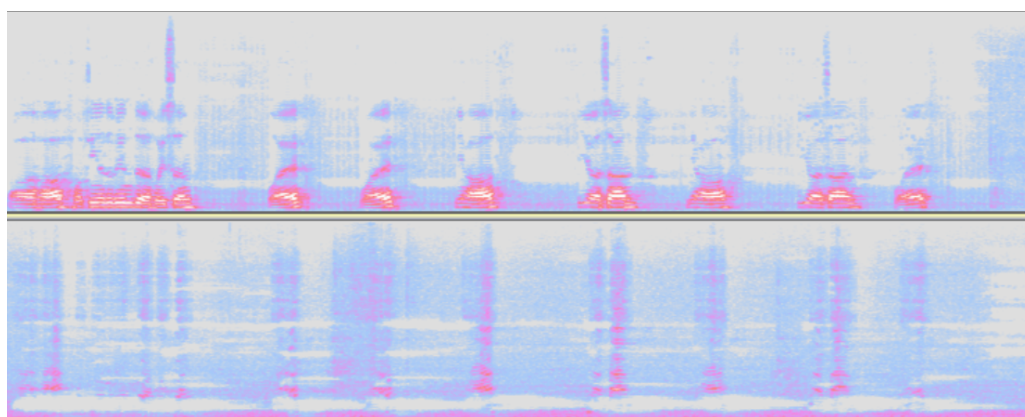
10. 喇叭声音破音、颤音

- a. 结构或喇叭原因
- b. 降低我方speaker的增益、降低对方mic的增益

11. AEC参数设置不当

a. 问题音档

如下图所示，音档依次为AecAoIn和AecOut。由图可知，经过AEC算法输出的音档还有一些回声残留，这是AEC强度设置过低导致。



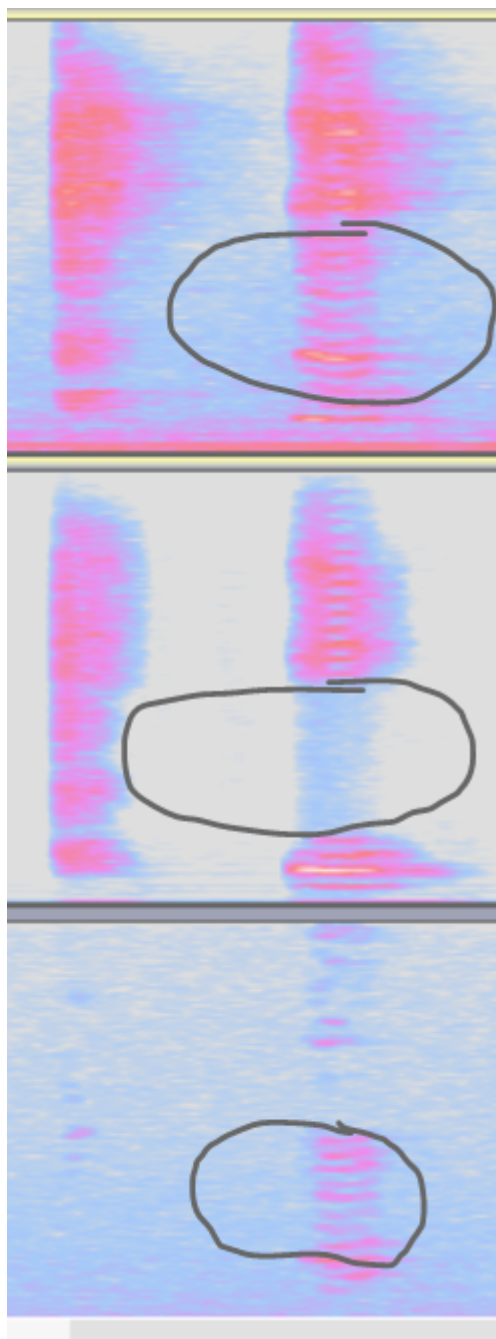
b. 解决办法

增加AEC强度。

12. Speaker失真导致AEC消不干净

a. 问题音档

如下图所示，音档依次为AecAiIn、AecAoIn和AecOut。可以看出由于speaker失真，使AecAiIn出现了AecAoIn中不存在的频段（图中标记部分所示），导致AEC算法不能获取正确的参考信号，使AEC算法消不干净，存在回声，存在回声的频段恰好是由于speaker失真产生的频段。



b. 解决办法

- i. 更换speaker
- ii. 减小Ao的增益

13. 电路问题引入噪声

a. 问题音档

如下图所示，这种存在明显规律的噪声一般是电路原因引起的。



b. 解决办法

请硬件工程师排查电路原因。

9. 注意事项

在speaker增益固定后，按当前的增益播放1KHz 0dB的正弦波音档或扫频音档，[TestFile3](#) [media/TestFile3_sweep.wav]，请硬件同事帮忙测量送到speaker端信号的峰峰值，根据speaker的阻抗，请硬件同事计算此增益下，功率是否超过了speaker本身的额定功率，以防烧坏喇叭。